

Deep I inzva tudy

Group

Week 5

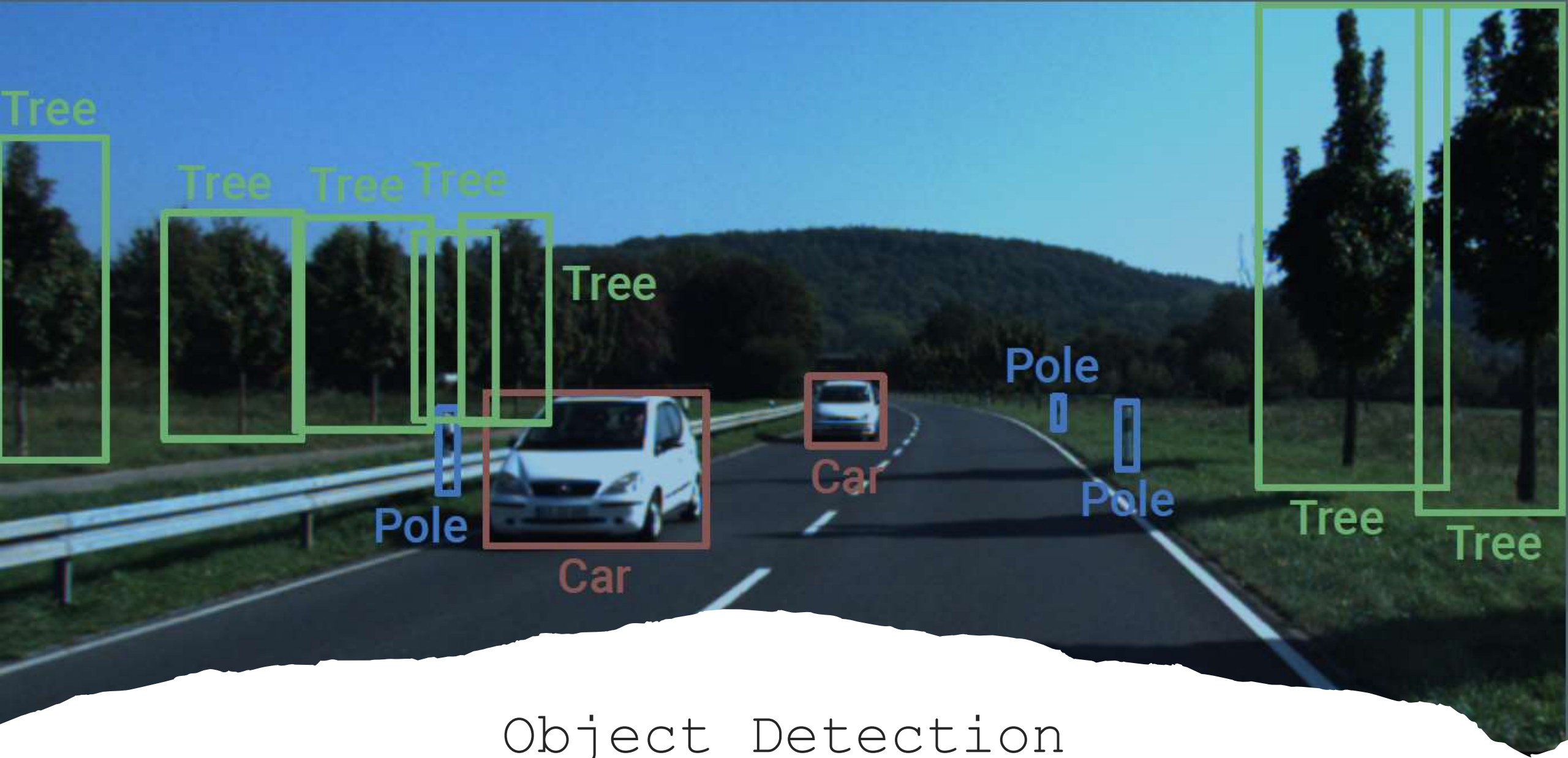
*Object Detection and
Image Segmentation*

Previous Week

- Convolution Operation
- Parameter Sharing
- Strided Convolution
- Padding
- Pooling
- Upsampling
- CNN Architectures: VGGNet, ResNet, Inception

Today's Computer Vision Tasks

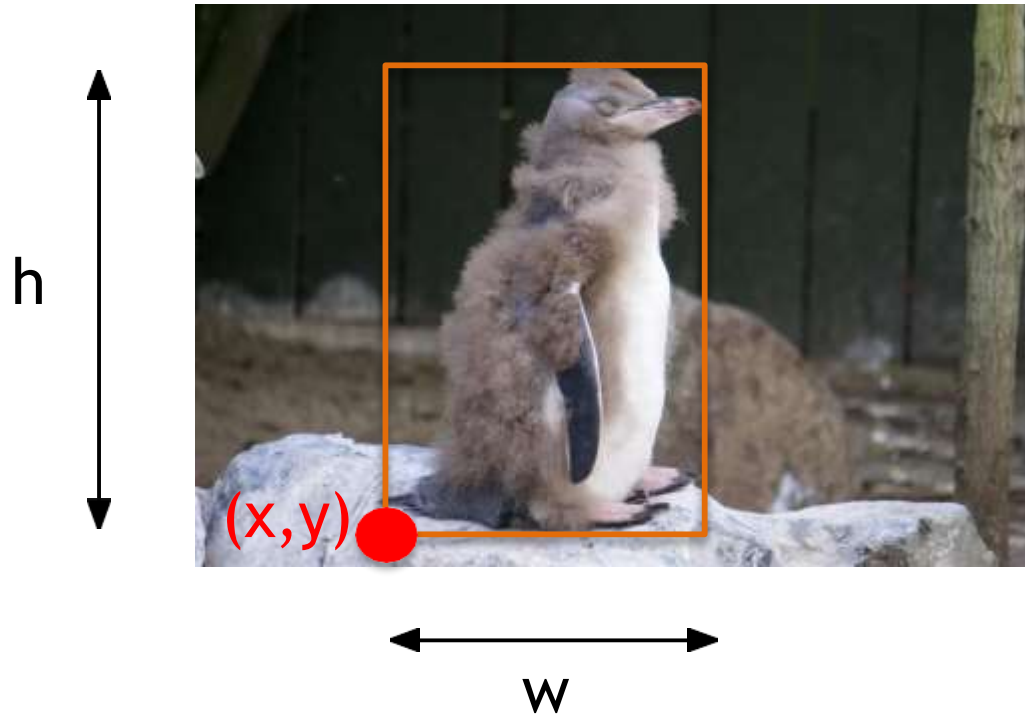
- Object Detection
- Semantic Segmentation
- Instance Segmentation



Object Detection

Task Definition of Object Detection

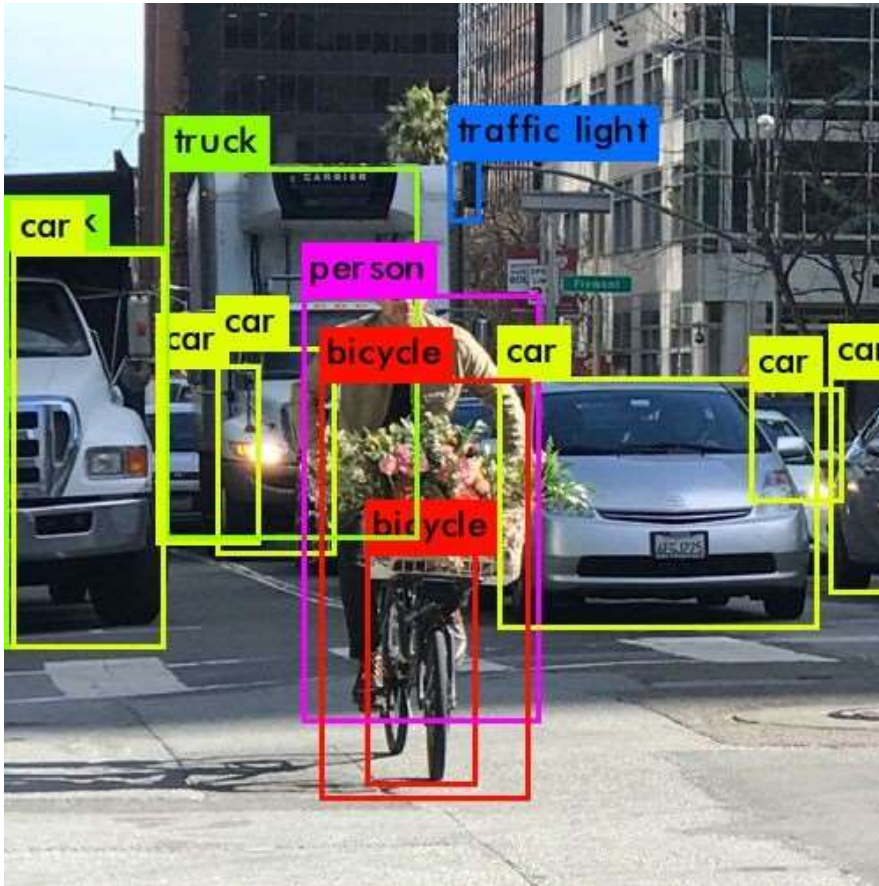
- Object detection problem



Bounding box. (x,y,w,h)

Task Definition of Object Detection

- Object detection problem



Bounding box. (x,y,w,h)

+
class

A bit of History of Object Detection

Traditional Object Detection Methods

- 1. Template matching + sliding window



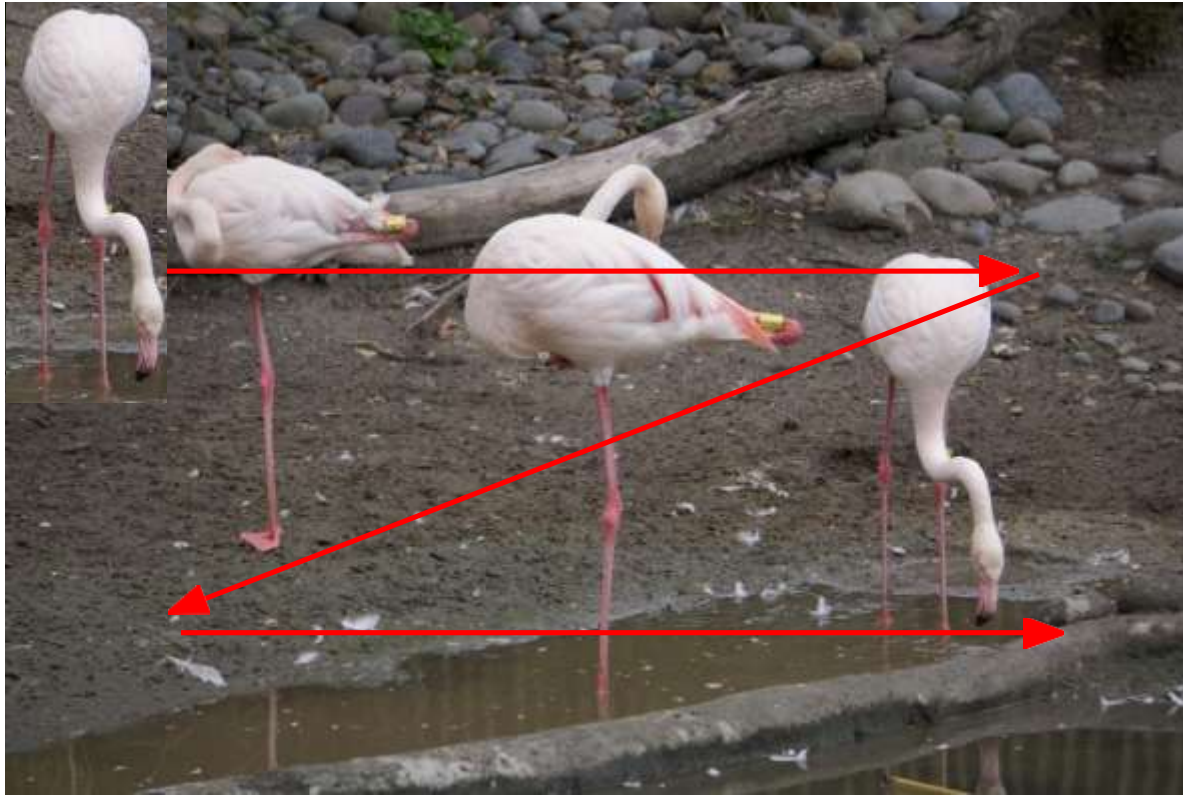
Image



Template

Traditional Object Detection Methods

- 1. Template matching + sliding window



Image



9

Template

Traditional object detection methods

- 1. Template matching + sliding window



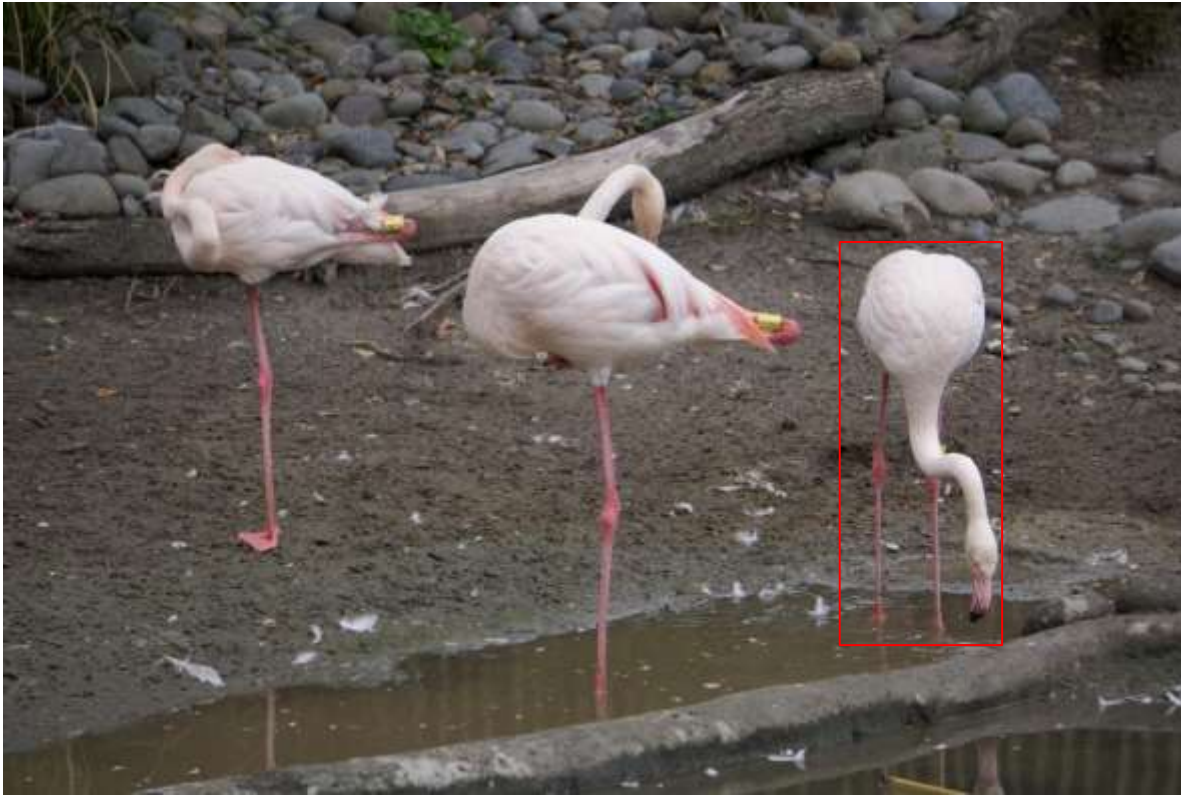
LOW
correlation

Image

For every position
you evaluate how
much do the pixels in
the image and
template correlate

Traditional object detection methods

- 1. Template matching + sliding window



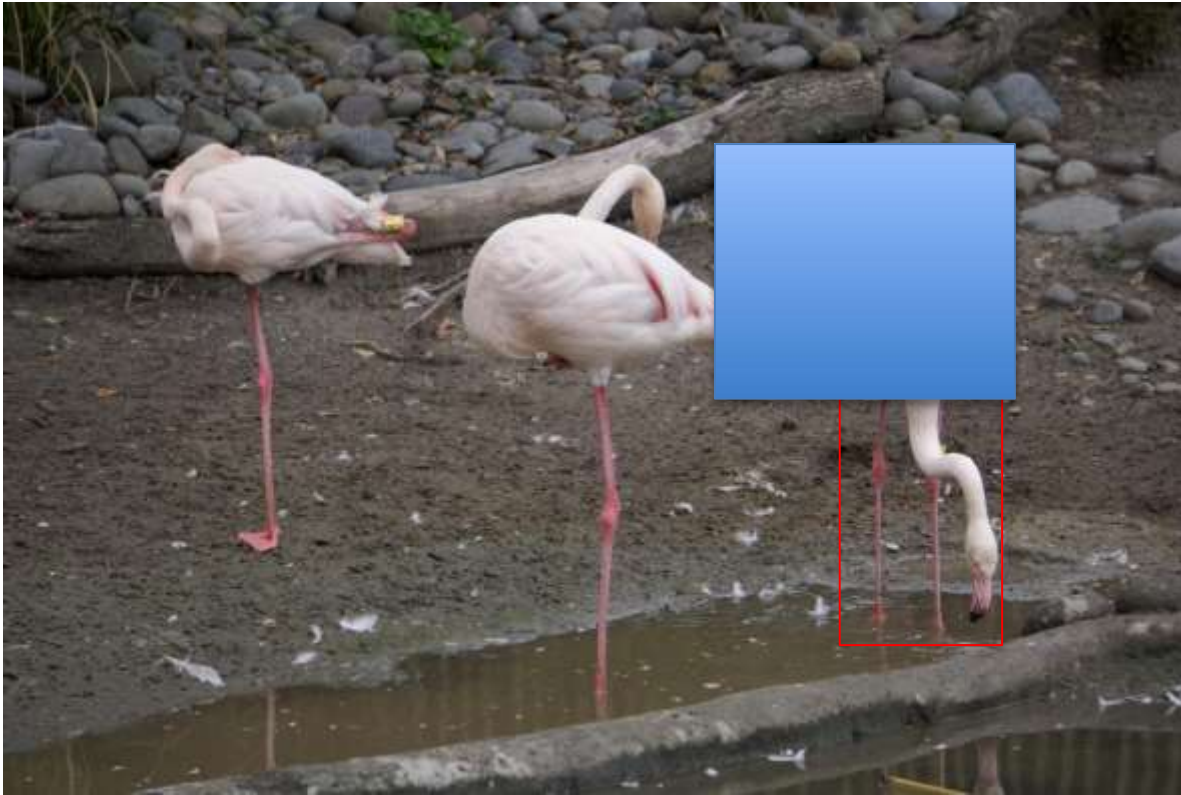
Image

HIGH
correlation

For every position
you evaluate how
much do the pixels in
the image and
template correlate

Traditional object detection methods

- Problems of 1. Template matching + sliding window



Image

LOW
correlation

For every position you evaluate how much do the pixels in the image and template correlate

1
2

Traditional object detection methods

- Problems of 1. Template matching + sliding window
 - Occlusions: we need to see the WHOLE object
 - This works to detect a given instance of an object but not a class of objects



Appearance and
shape changes



Pose changes

Traditional object detection methods

- Problems of 1. Template matching + sliding window
 - Occlusions: we need to see the WHOLE object
 - This works to detect a given instance of an object but not a class of objects
 - Objects have an unknown position, scale and aspect ratio, the search space is searched inefficiently with sliding window

Traditional object detection methods

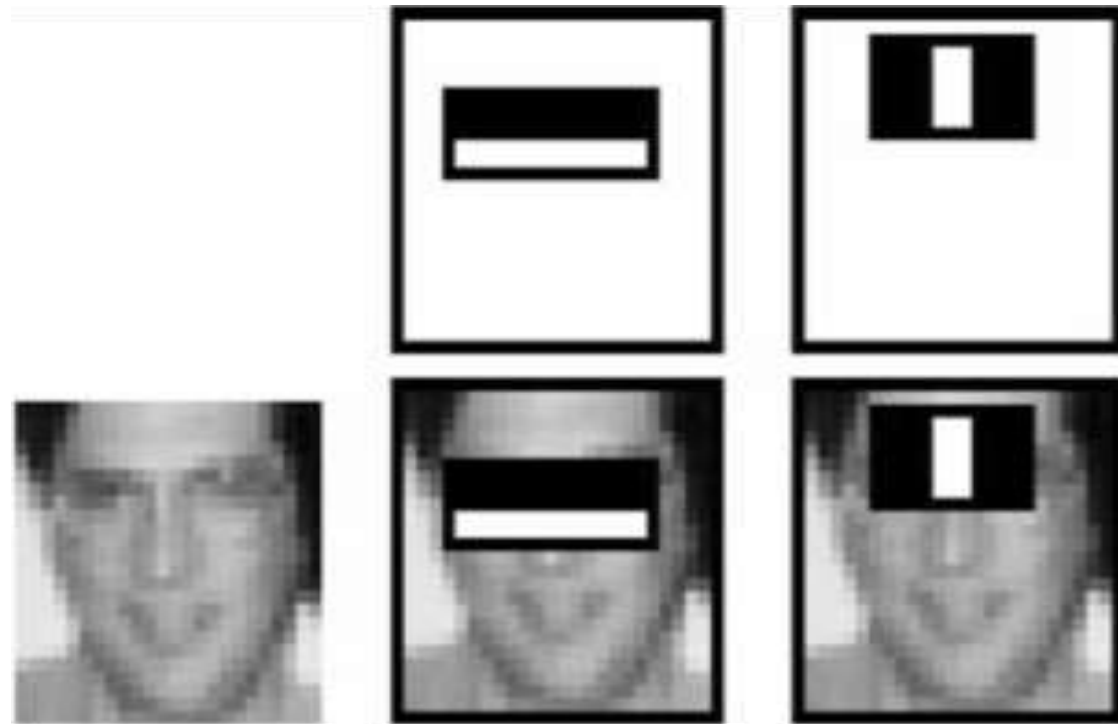
- **Feature extraction + classification**

Viola-Jones detector

- 2. Feature extraction + classification
 - Learning multiple weak learners to build a strong classifier
 - That is, make many small decisions and combine them for a stronger final decision

Viola-Jones detector

- 2. Feature extraction + classification



Haar features

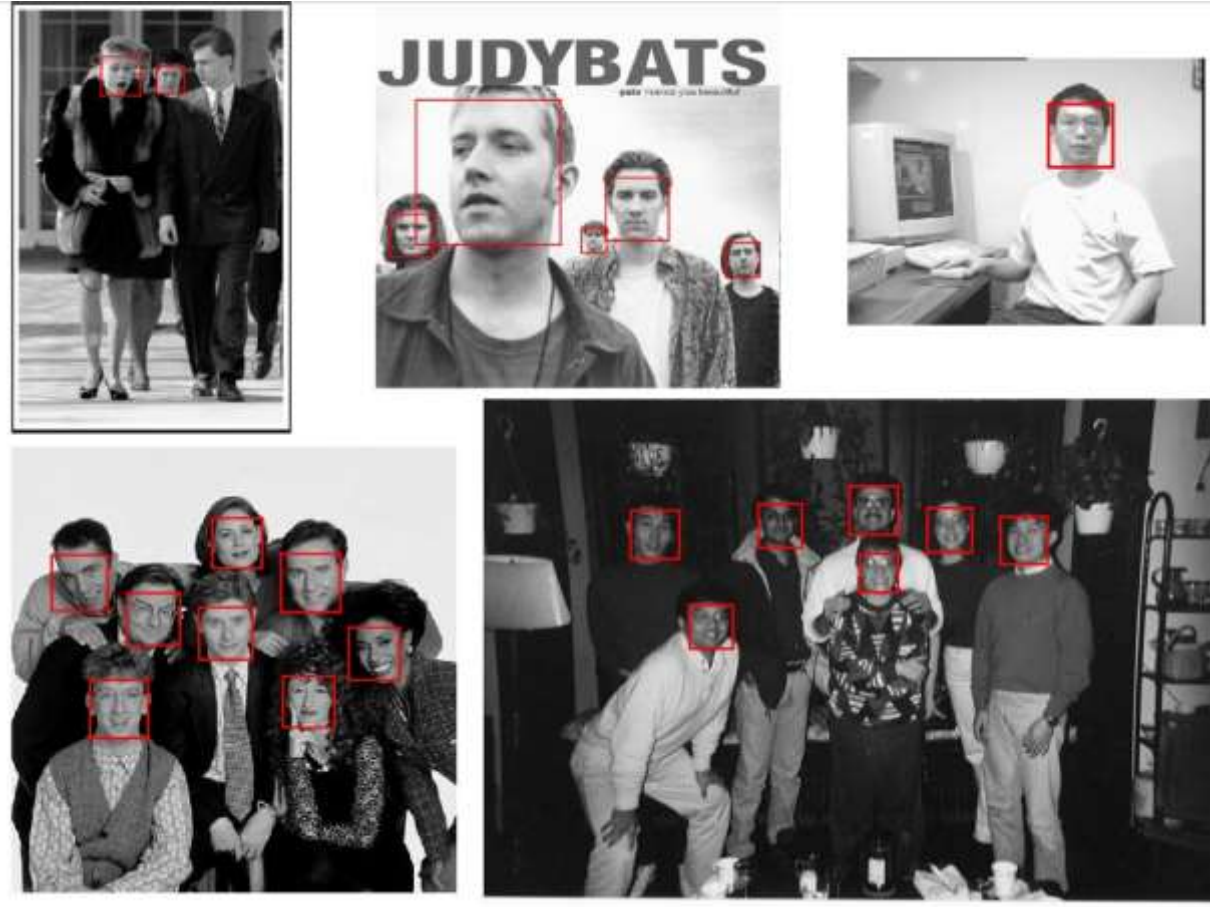
1
7

Viola-Jones detector

- 2. Feature extraction + classification
 - Step 1: Select your Haar-like features
 - Step 2: Integral image for fast feature evaluation
 - I can evaluate which parts of the image have highest cross-correlation with my feature (template)
 - Step 3: AdaBoost for to find weak learner
 - I cannot possibly evaluate all features at test time for all image locations
 - Learn the best set of weak learners
 - Our final classifier is the linear combination of all weak learners

1
8

Viola-Jones detector

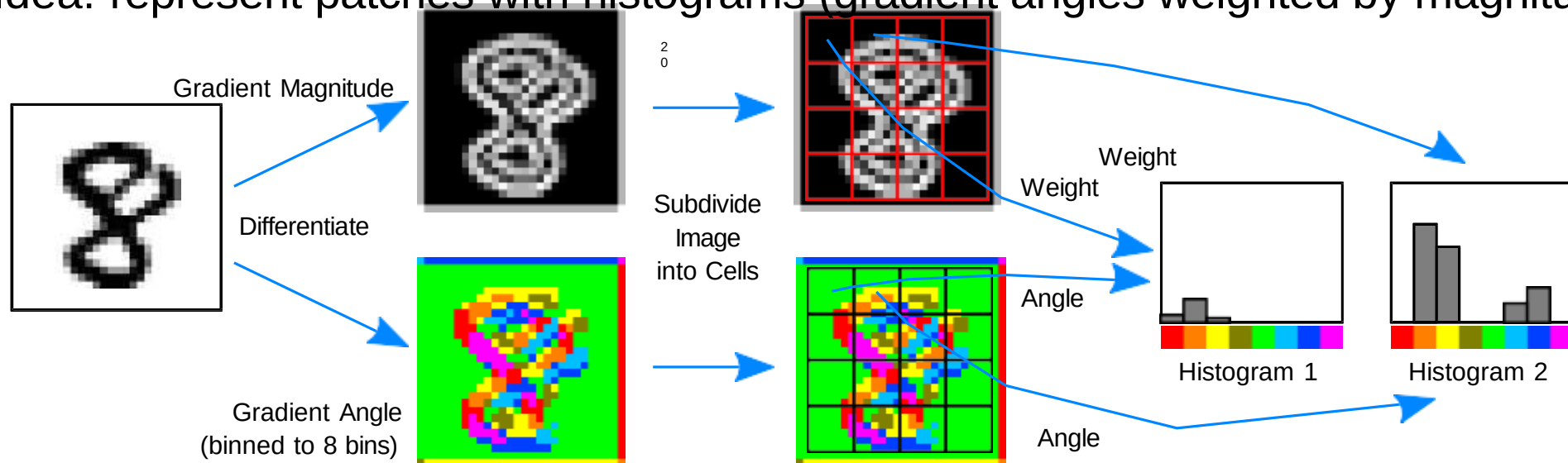


Viola and Jones. Rapid object detection using a boosted cascade of simple features. CVPR 2001.

Histograms of Oriented Gradients

Which feature space to use?

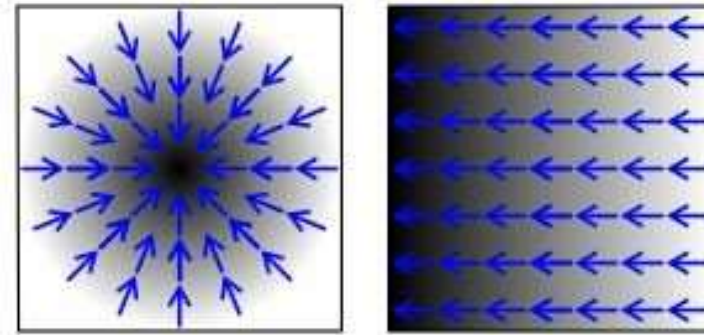
- ▶ Simplest: RGB pixel space $\Rightarrow$ neither viewpoint nor illumination invariant
- ▶ Better: Histograms of Oriented Gradients (HoG) (defacto standard for >10 years)
 - ▶ Idea: represent patches with histograms (gradient angles weighted by magnitude)



- ▶ HoG is invariant to small deformations (translation, scale, rotation, perspective)

Histogram of Oriented Gradients

- 2. Feature extraction + classification



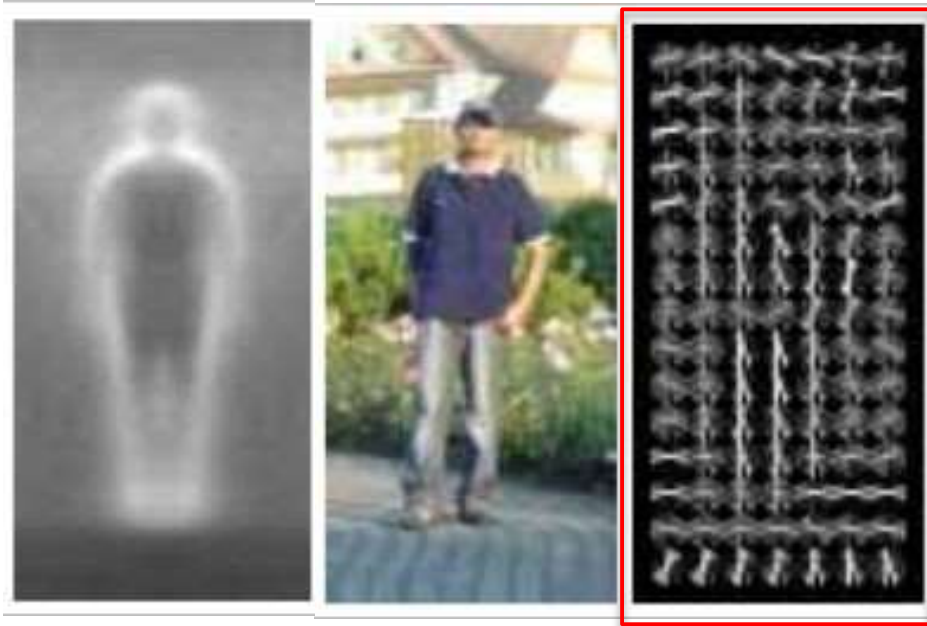
Gradient: blue arrows show the gradient, i.e., the direction of greatest change of the image.

2
1

Average gradient image over training samples □ gradients provide shape information. Let us create a descriptor that exploits that.

Histogram of Oriented Gradient

- 2. Feature extraction + classification



HOG descriptor □ Histogram of oriented gradients.

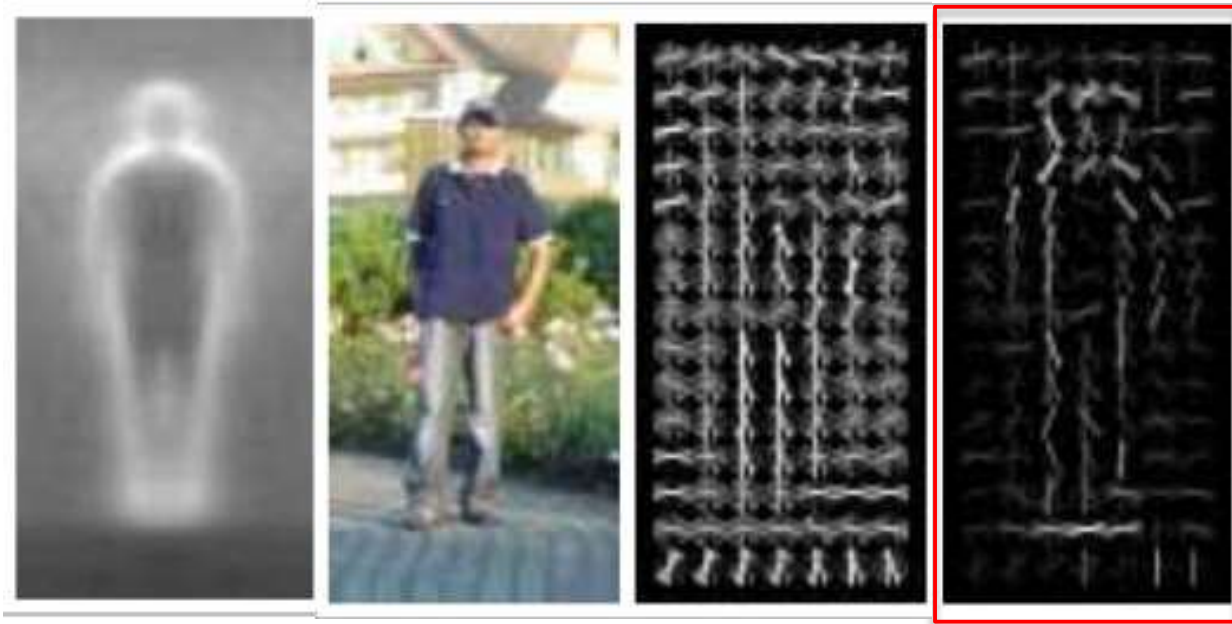
Compute gradients in dense grids, compute gradients and create a histogram based on gradient direction.

Histogram of Oriented Gradients

- 2. Feature extraction + classification
 - Step 1: Choose your training set of images that contain the object you want to detect.
 - Step 2: Choose a set of images that do NOT contain that object.
 - Step 3: Extract HOG features on both sets.
 - Step 4: Train an SVM classifier on the two sets to detect whether a feature vector represents the object of interest or not (0/1 classification).

Histogram of Oriented Gradients

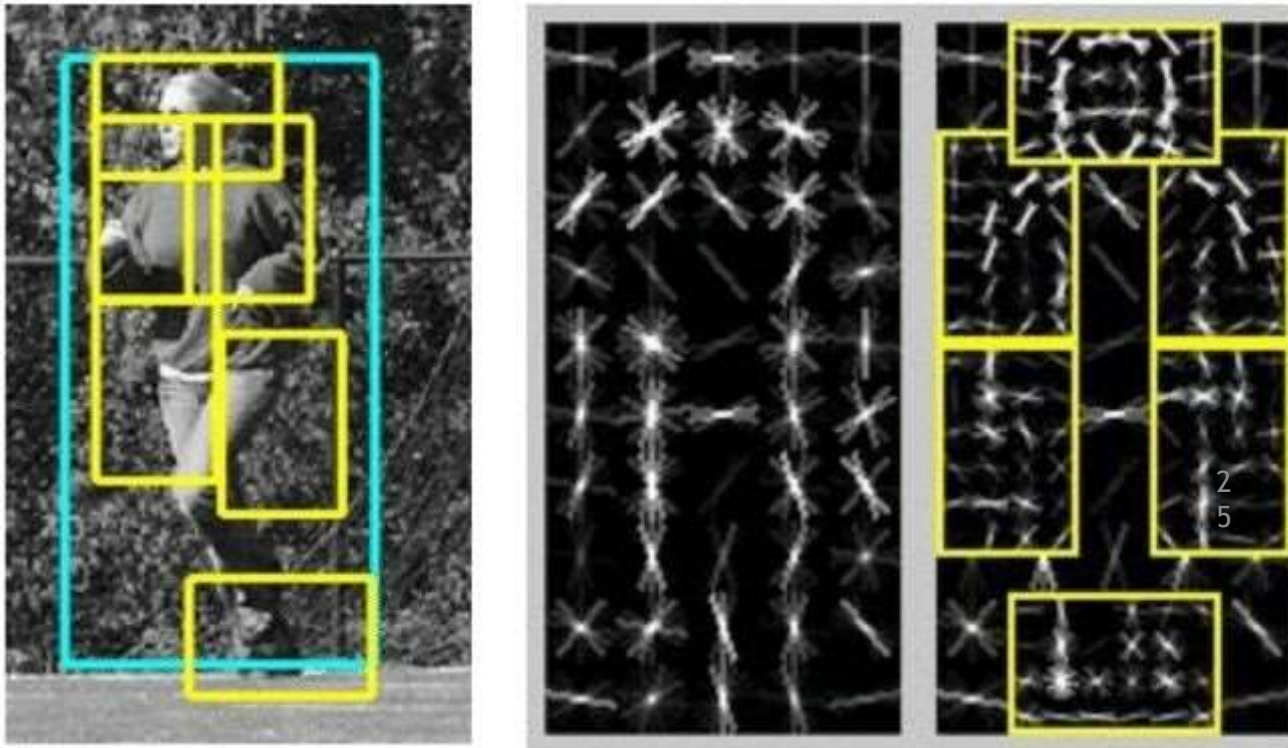
- 2. Feature extraction + classification



HOG features weighted by the positive SVM weights - the ones used for the pedestrian object classifier.

Deformable Part Model

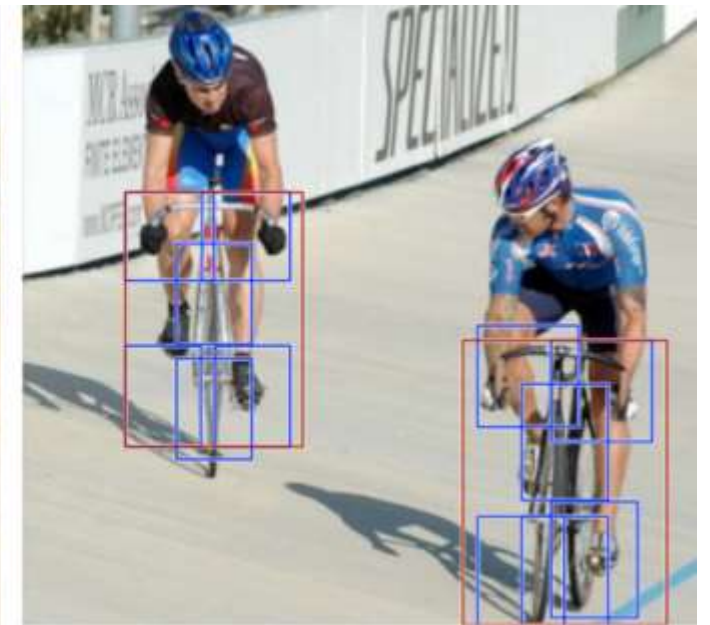
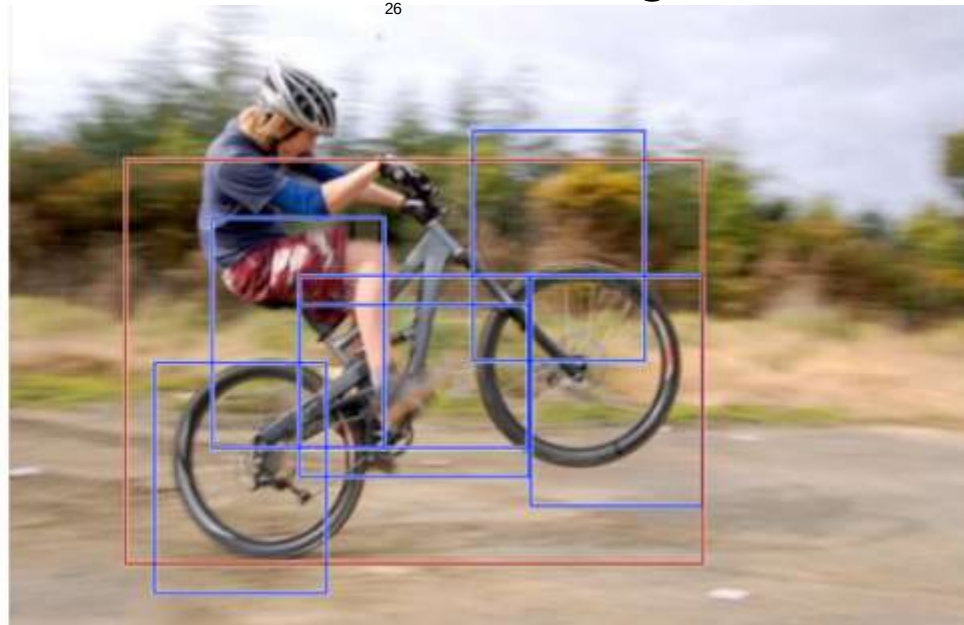
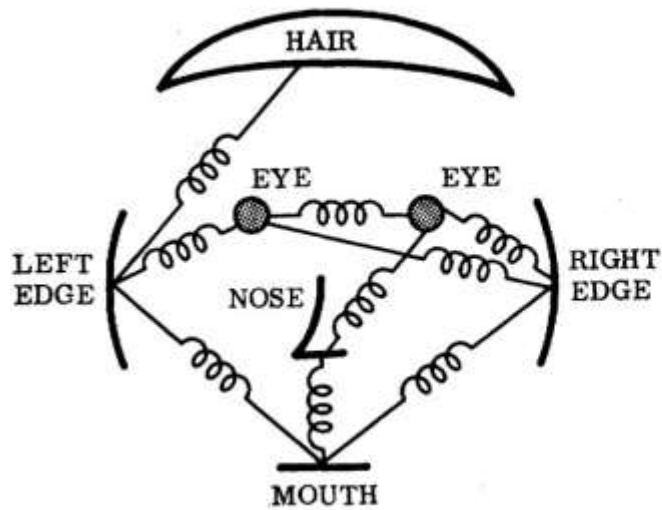
- Also based on HOG features, but based on body part detection $\square$ more robust to different body poses



Part Based Models

Part-based Models:

- ▶ Idea: Model object based on its parts, **model distribution of part configurations**
- ▶ Allows for even more invariance (non-rigid deformations etc.)
- ▶ However: **slower inference** and not much gain wrt. multi-view HoG models



Part Based Models

- Instead of modeling an object as a whole (like a single bounding box template), part-based models:

- **Decompose the object into meaningful parts**

- Example:**

- Human → head, torso, arms, legs
 - Car → wheels, windows, body
 - Face → eyes, nose, mouth

- **Model the appearance of each part separately**

- **Model the spatial configuration between parts**
(i.e., how parts are arranged relative to each other)

- So the object is defined by:

- **Part appearance models**

- **Geometric constraints between parts**

- **A probabilistic distribution over part configurations**

Why Use Part-Based Models?

Objects in real images:

- Change pose
- Deform (e.g., humans, animals)
- Get partially occluded
- Appear at different scales

A rigid template struggles with these variations.

Part-based models handle this by allowing:

- Flexible movement between parts
- Some parts to be missing (occlusion handling)
- Deformation modeling

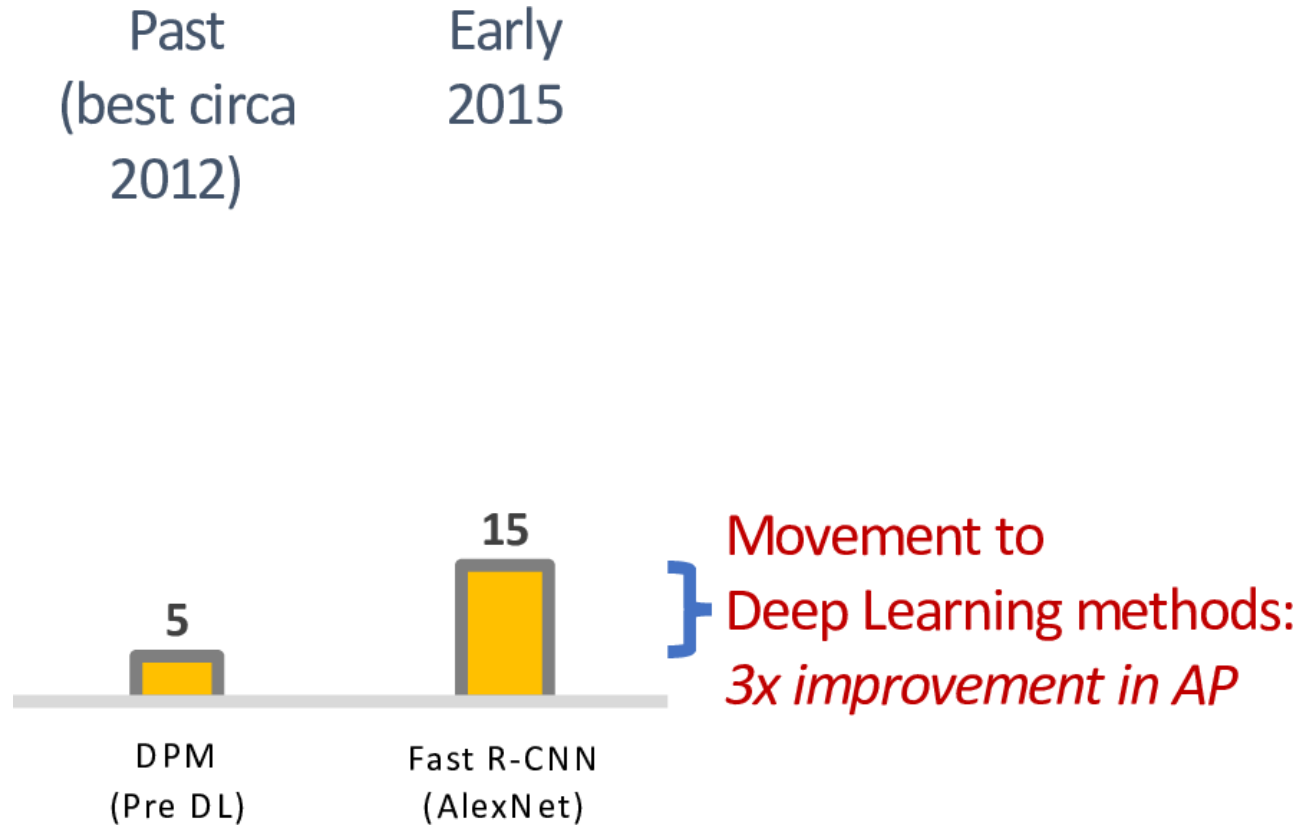
Recognition – A Success Story

- Classical approaches **haven't lead to major breakthroughs:** only
- non-safety-critical tasks (e.g., face detection), they h
- **Why?**

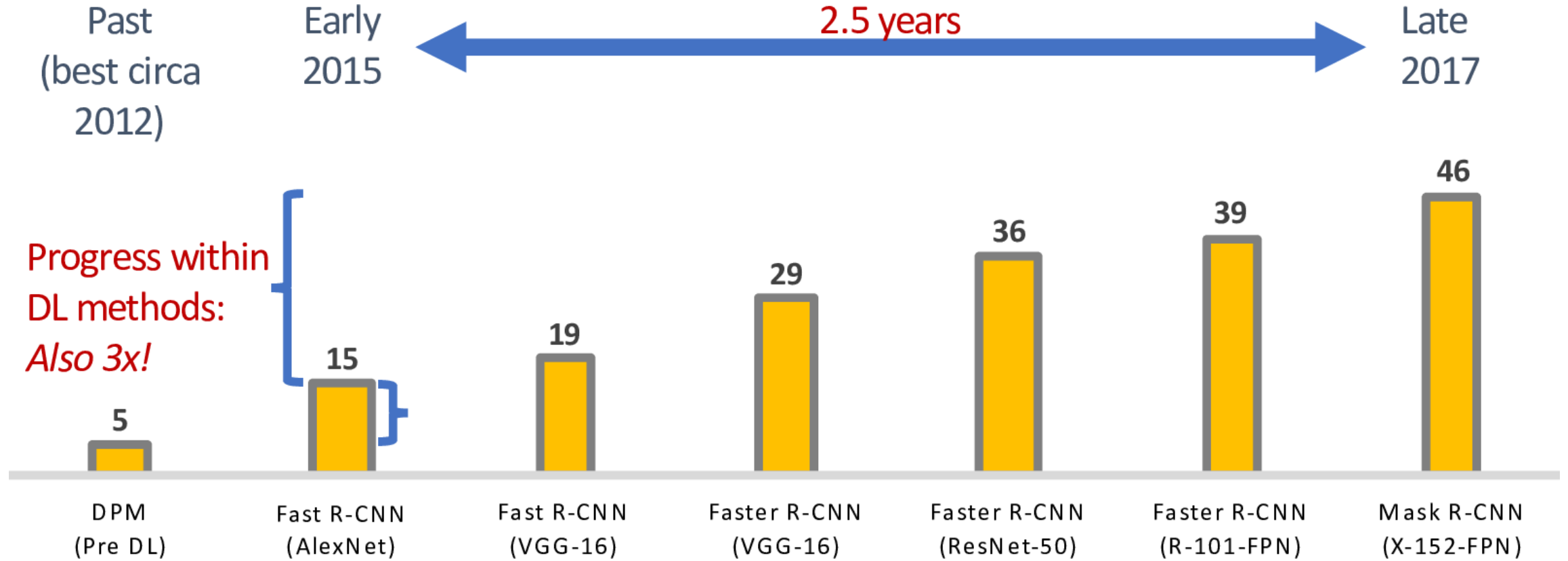
29

- ▶ Computer vision features are very difficult to hand-engineer
- ▶ It is unclear how a good representation should look like
- ▶ Furthermore, sliding window approaches are slow in practice
- ▶ Inference in complicated part-based models is even slower
- ▶ Computation is often not efficiently re-used
- ▶ But: deep learning has changed this ..

Hand-crafted Representations vs. Learned Representations



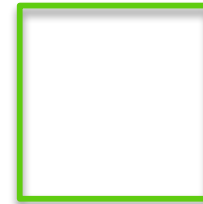
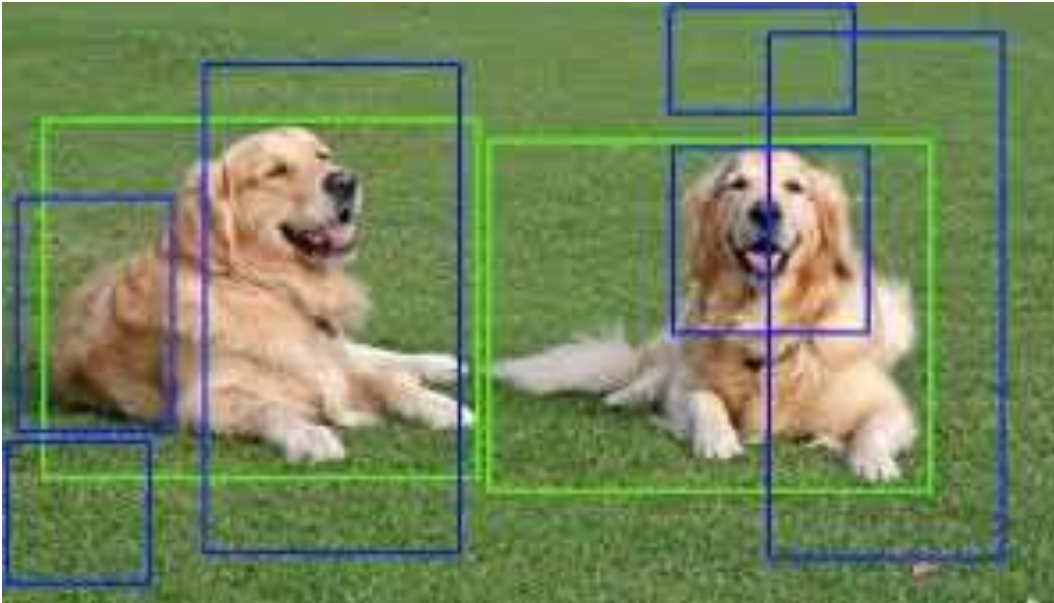
Hand-crafted Representations vs. Learned Representations



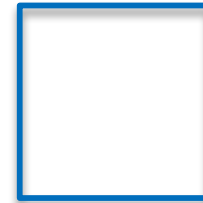
How to move toward
general object
detection?

What defines an object?

- We need a generic, class-agnostic objectness measure:
how likely it is for an image region to contain an object



- Very likely to be an object

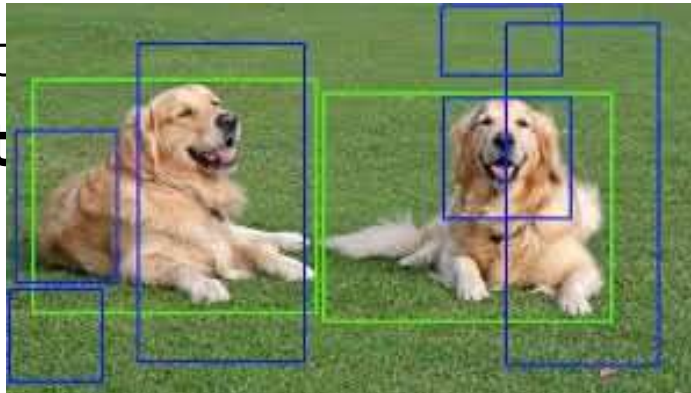


- Maybe it is an object

What defines an object?

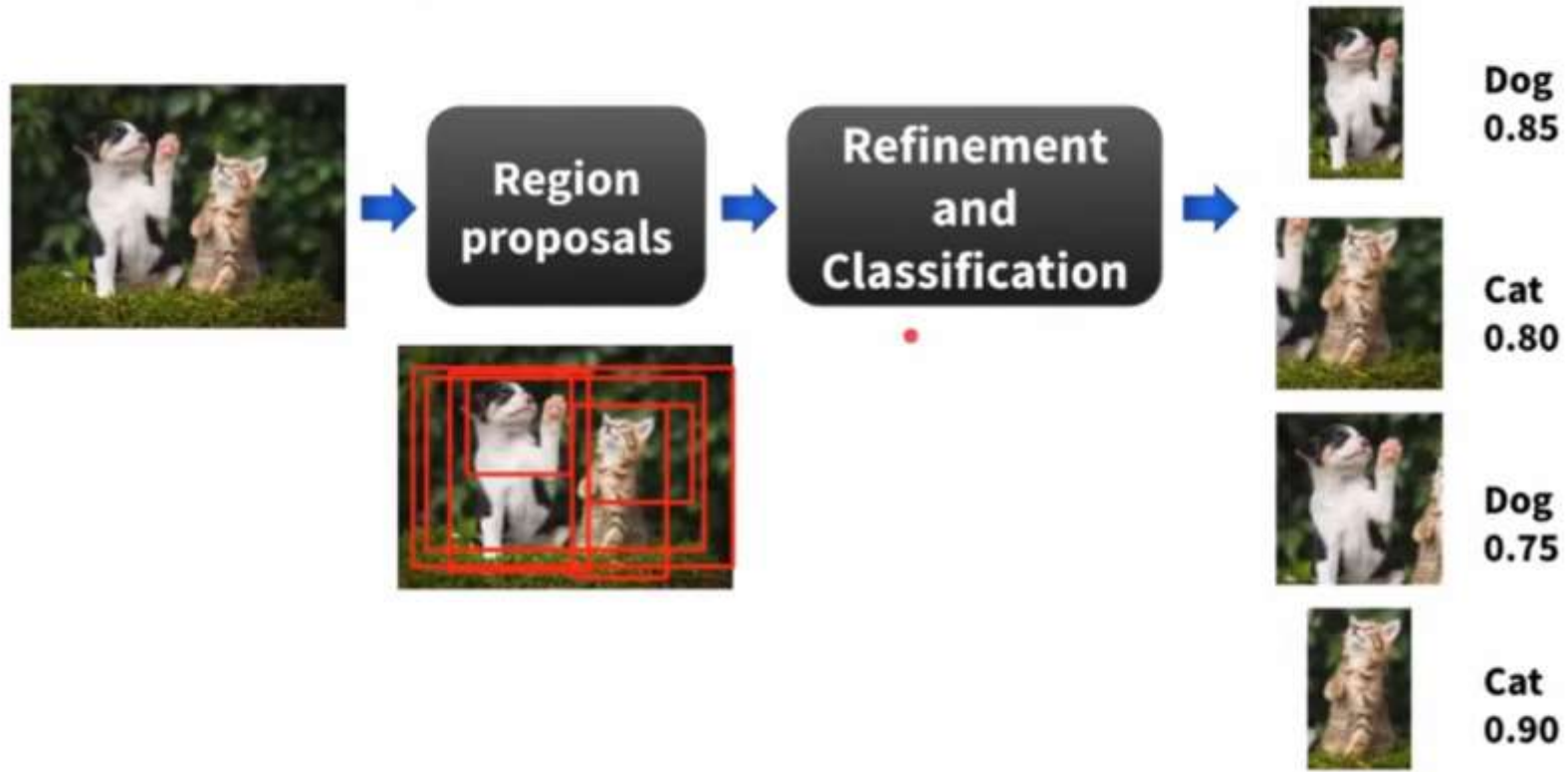
- We need a generic, class-agnostic objectness measure:
how likely it is for an image region to contain an object

- Using this measure yields a number of candidate **regions** or **regions of interest** for focus.



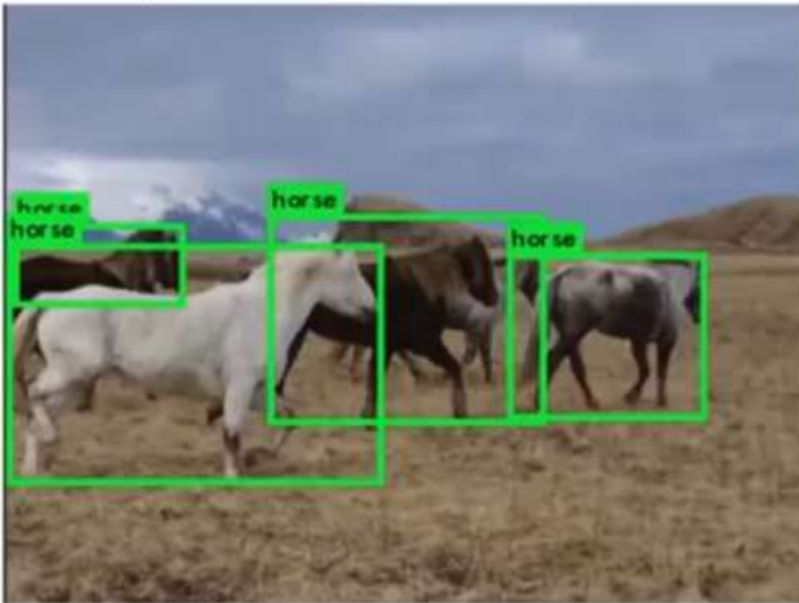
+
classifier

Region Proposals

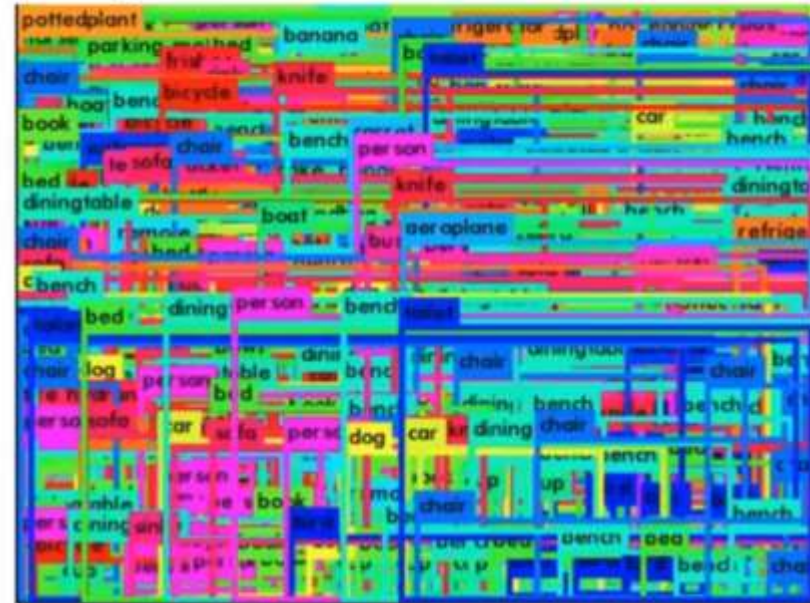


Region Proposals

We expect:



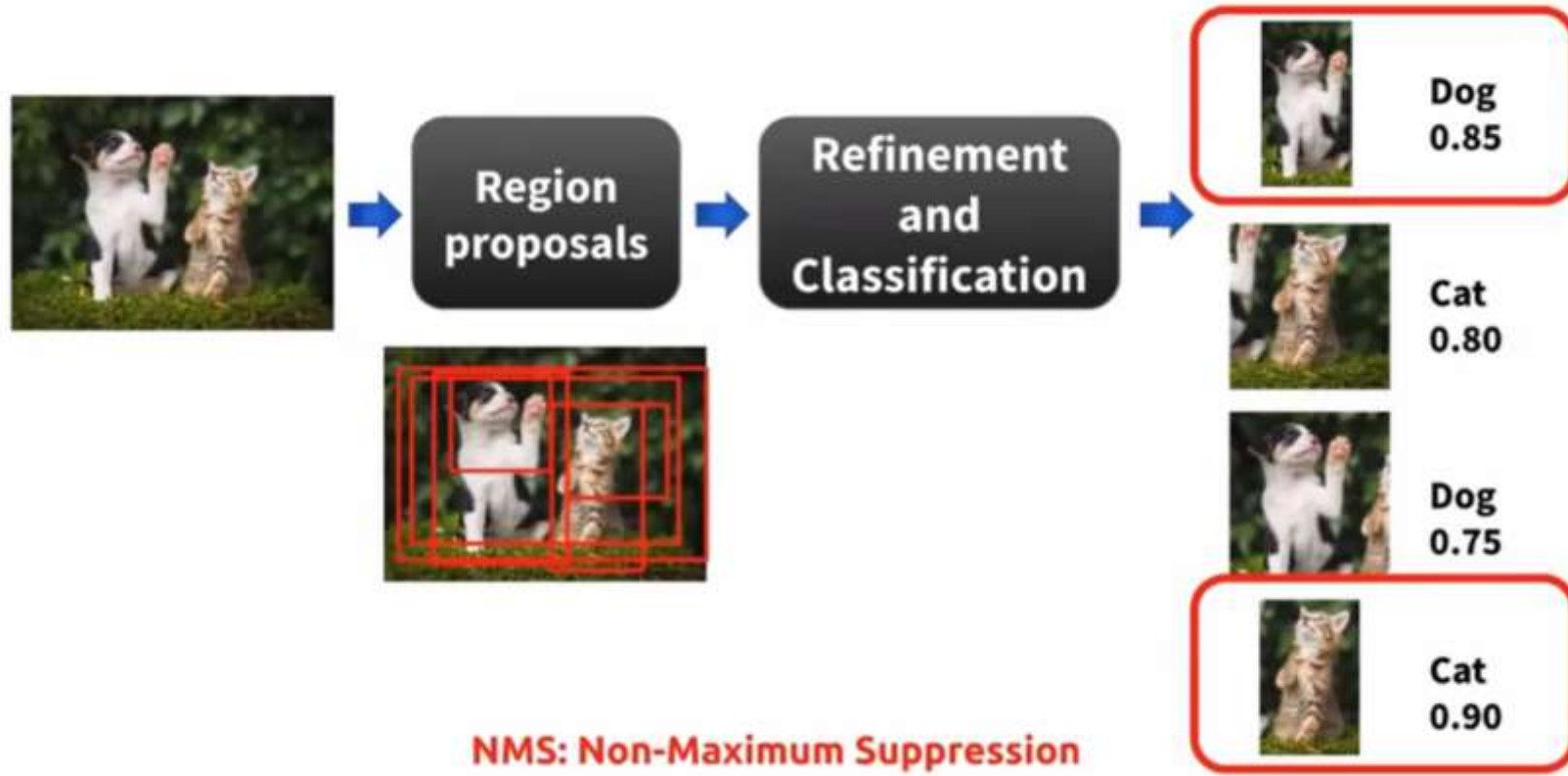
We get:



How can the multiple detections be filtered out?

Non Maximum Suppression & Threshold over the classification score.

Region Proposals + NMS



Object Proposal Methods

- Selective search: van de Sande et al. Segmentation as selective search for object recognition. ICCV 2011.
- Edge boxes: Zitnick and Dollar. Edge boxes: locating object proposals from edges. ECCV 2014.

Object proposal methods: **Selective Search**



Input Image

□ How does it work?

1.Initial segmentation (superpixels)

- 1.The image is divided into small regions (superpixels)

2.Region merging (hierarchical grouping)

- 1.Similar regions are merged based on:
 - 1.Color similarity
 - 2.Texture similarity
 - 3.Size
 - 4.Fill (how well regions fit together)

3.Multi-scale search

- 1.The process is repeated with different

Object proposal methods:

Edge Boxes

"Edge Boxes score bounding boxes based on the number of enclosed edges, assuming objects contain many internal edges."

□ How Does Edge Boxes Work?

1. Edge Detection

Extract edge information from the image using an edge detector.

2. Box Scoring

Assign a score to each bounding box based on:

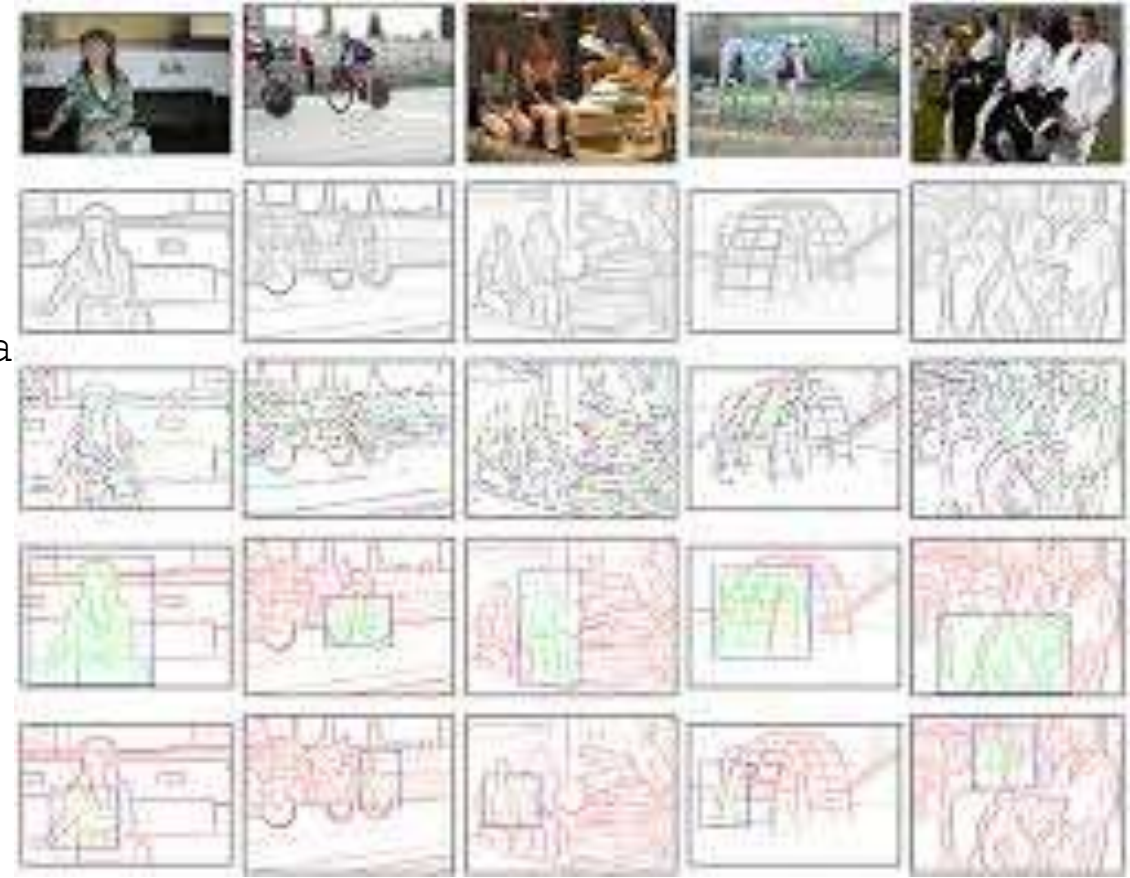
- The number of edges **inside** the box minus the edges that **cross the boundary**

3. Search Over Boxes

Evaluate a large number of candidate boxes at different **locations and scales**.

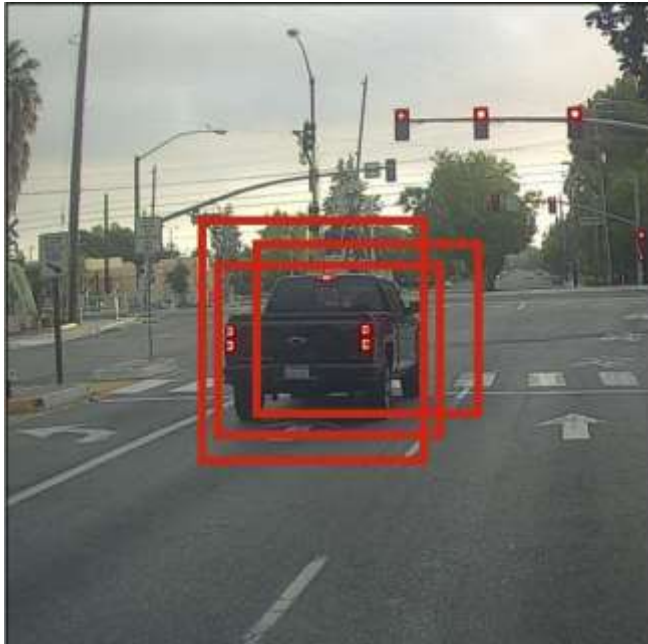
4. Selection

Select the **top-scoring bounding boxes** as object proposals.



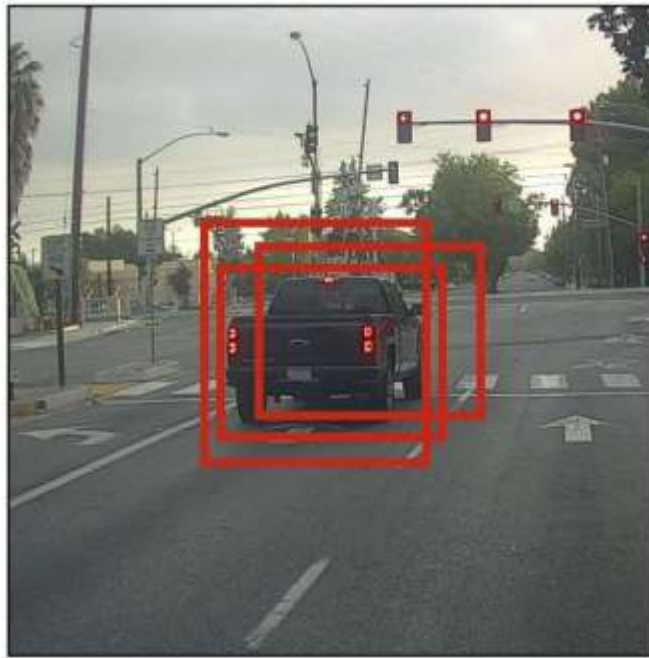
Do we want all proposals?

- Many boxes trying to explain one object
- We need a method to keep only the “best” boxes



Non-Maximum Suppression (NMS)

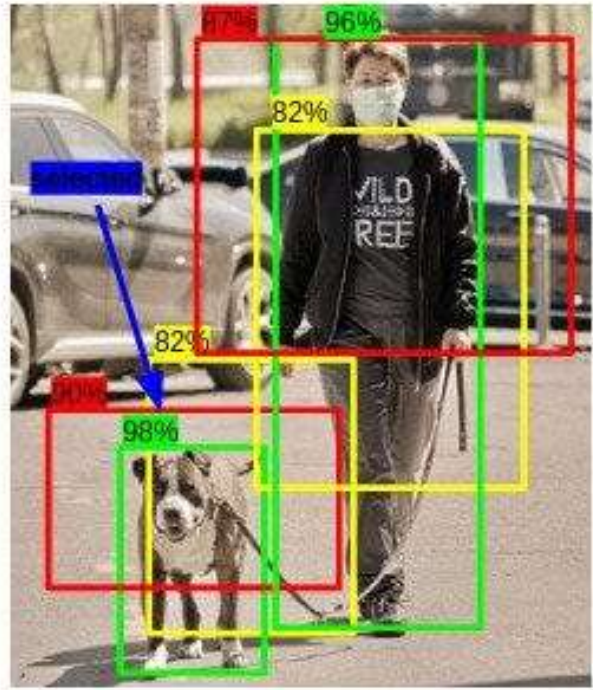
- Many boxes trying to explain one object
- We need a method to keep only the “best” boxes



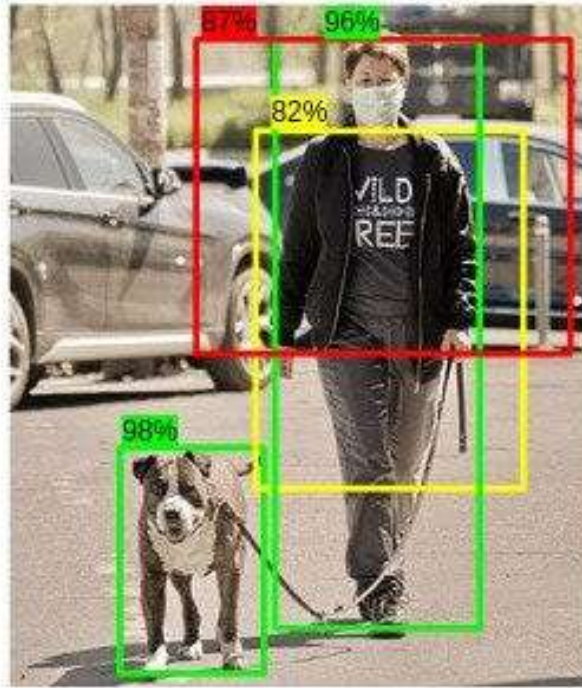
Non-Max
Suppression



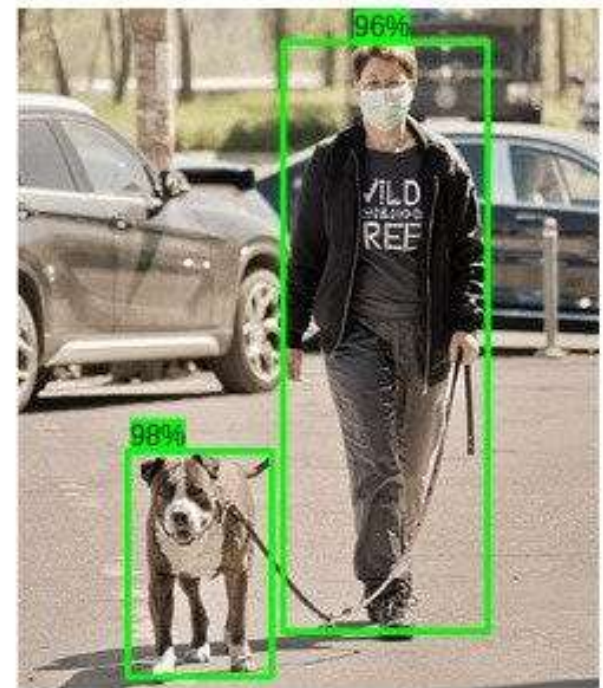
Non-Maximum Suppression (NMS)



Step 1: Selecting Bounding box with highest score



Step 3: Delete Bounding box with high overlap



Step 5: Final Output

Non-Maximum Suppression (NMS)

Algorithm 1 Non-Max Suppression

```
1: procedure NMS( $B, c$ )
2:    $B_{nms} \leftarrow \emptyset$ 
3:   for  $b_i \in B$  do  Start with anchor box i
4:      $discard \leftarrow \text{False}$ 
5:     for  $b_j \in B$  do  For another box j
6:       if  $\text{same}(b_i, b_j) > \lambda_{nms}$  then  If they overlap
7:         if  $\text{score}(c, b_j) > \text{score}(c, b_i)$  then
8:            $discard \leftarrow \text{True}$   Discard box i if the
9:         if not  $discard$  then score is lower than
10:           $B_{nms} \leftarrow B_{nms} \cup b_i$  the score of j
11:   return  $B_{nms}$ 
```

CV3DST | Prof. Leal-Taixé

4

4

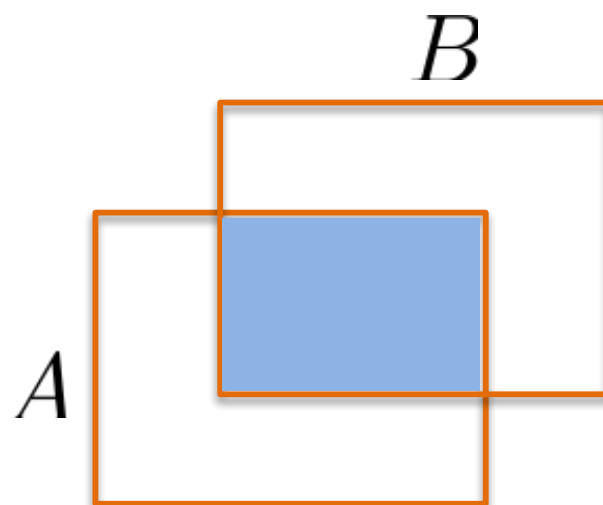
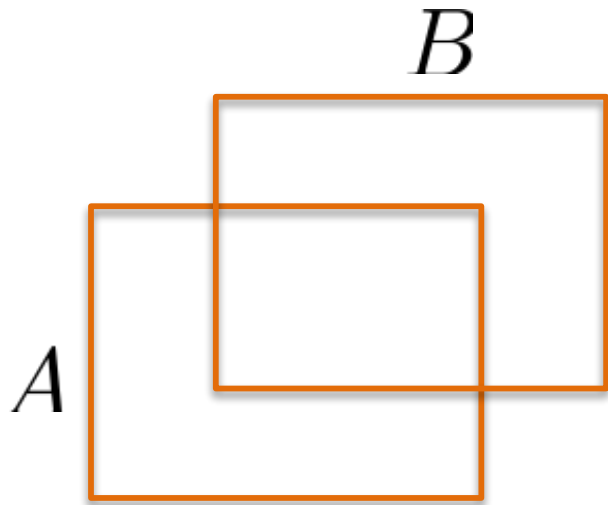
Overlap = to be defined

Score = depends on the task

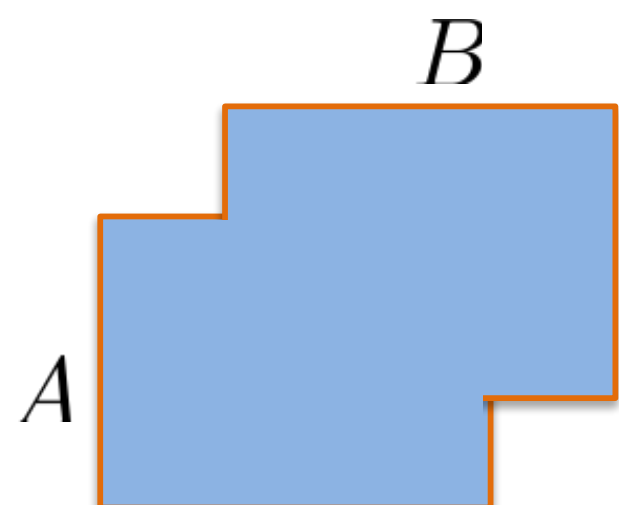
Region overlap

- We measure region overlap with the Intersection over Union (IoU) or Jaccard Index:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$



Intersection



Union

Non-Maximum Suppression (NMS)

Algorithm 1 Non-Max Suppression

```
1: procedure NMS( $B, c$ )
2:    $B_{nms} \leftarrow \emptyset$ 
3:   for  $b_i \in B$  do
4:      $discard \leftarrow \text{False}$ 
5:     for  $b_j \in B$  do
6:       if  $\text{same}(b_i, b_j) > \lambda_{nms}$  then
7:         if  $\text{score}(c, b_j) > \text{score}(c, b_i)$  then
8:            $discard \leftarrow \text{True}$ 
9:       if not  $discard$  then
10:         $B_{nms} \leftarrow B_{nms} \cup b_i$ 
11:  return  $B_{nms}$ 
```

Start with anchor box i

For another box j

If they overlap

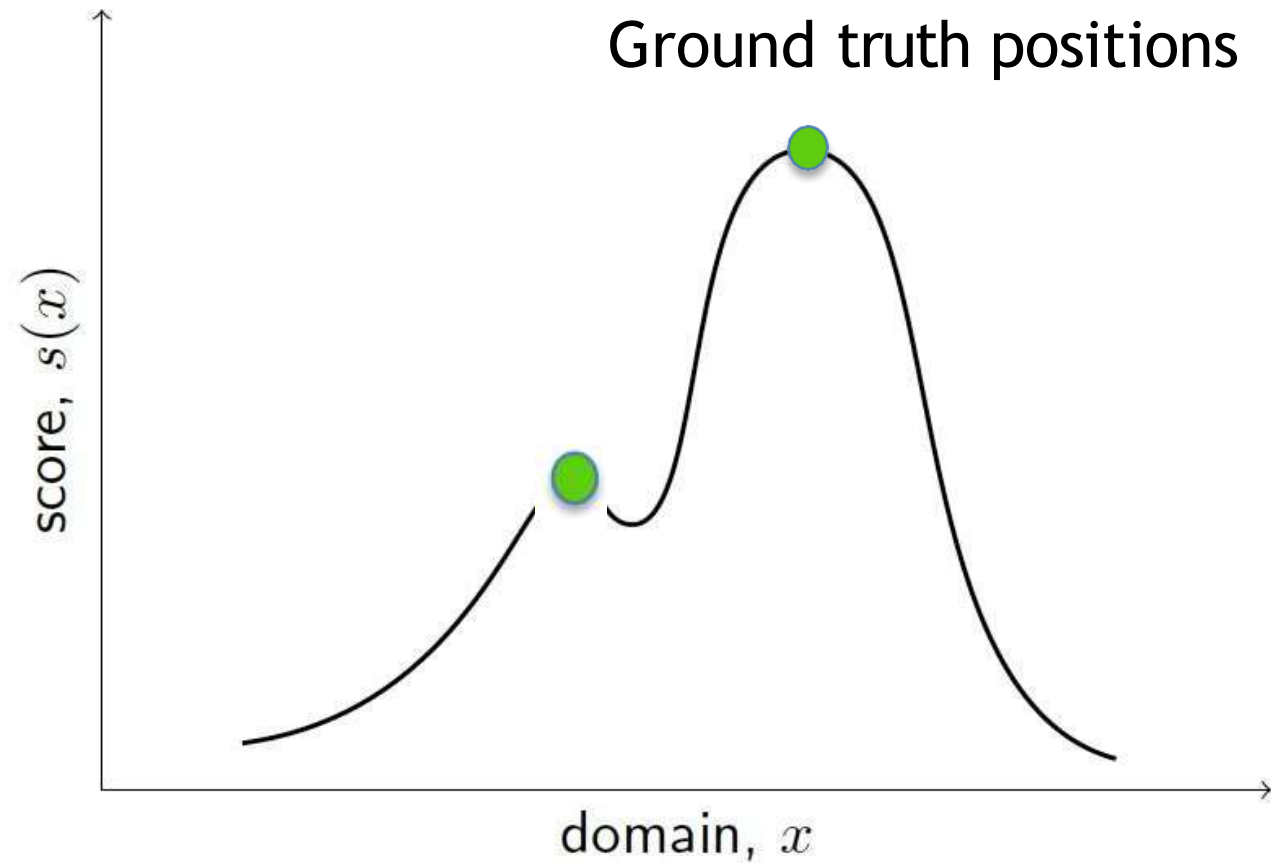
Discard box i if the score is lower than the score of j

4
6

Overlap = to be defined

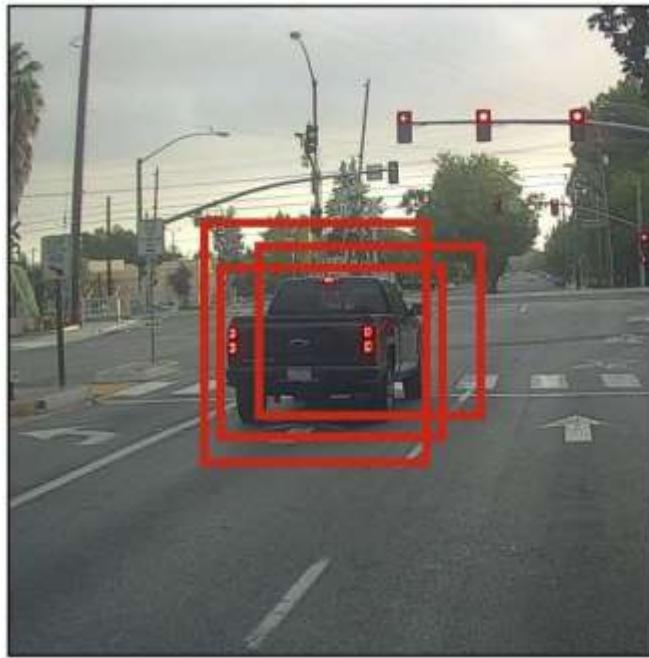
Score = depends on the task

NMS: the problem



Non-Maximum Suppression (NMS)

- NMS will be used at test time. Most detection methods (even Deep Learning ones) use NMS!



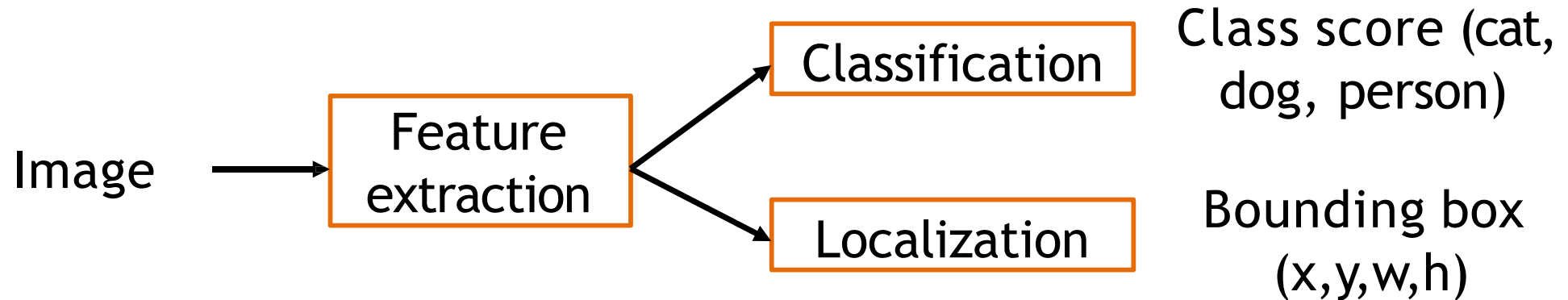
Non-Max
Suppression



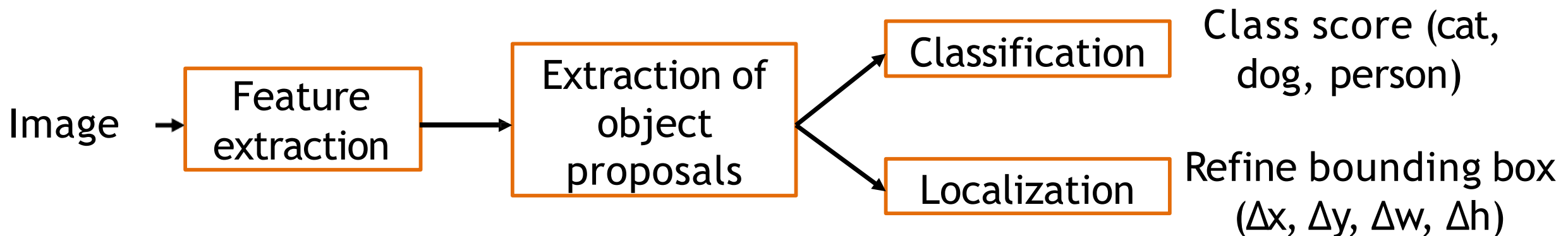
Learning Based Detectors

Types of object detectors

- One-stage detectors



- Two-stage detectors



Types of object detectors

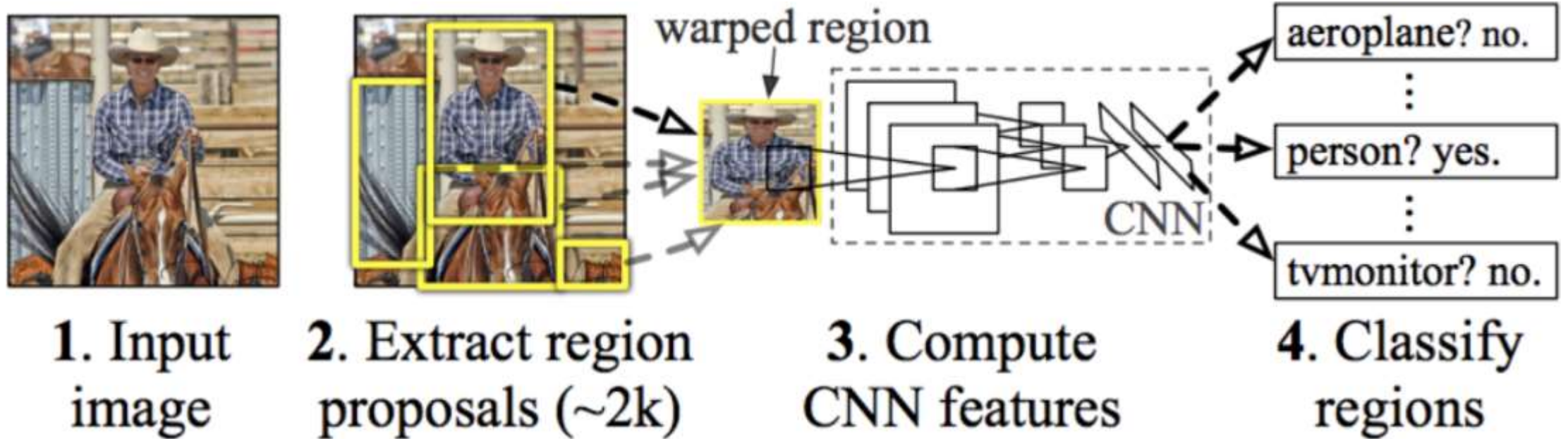
- One-stage detectors
 - YOLO, SSD
- Two-stage detectors
 - R-CNN, Fast R-CNN, Faster R-CNN
 - SPP-Net, R-FCN, FPN

51

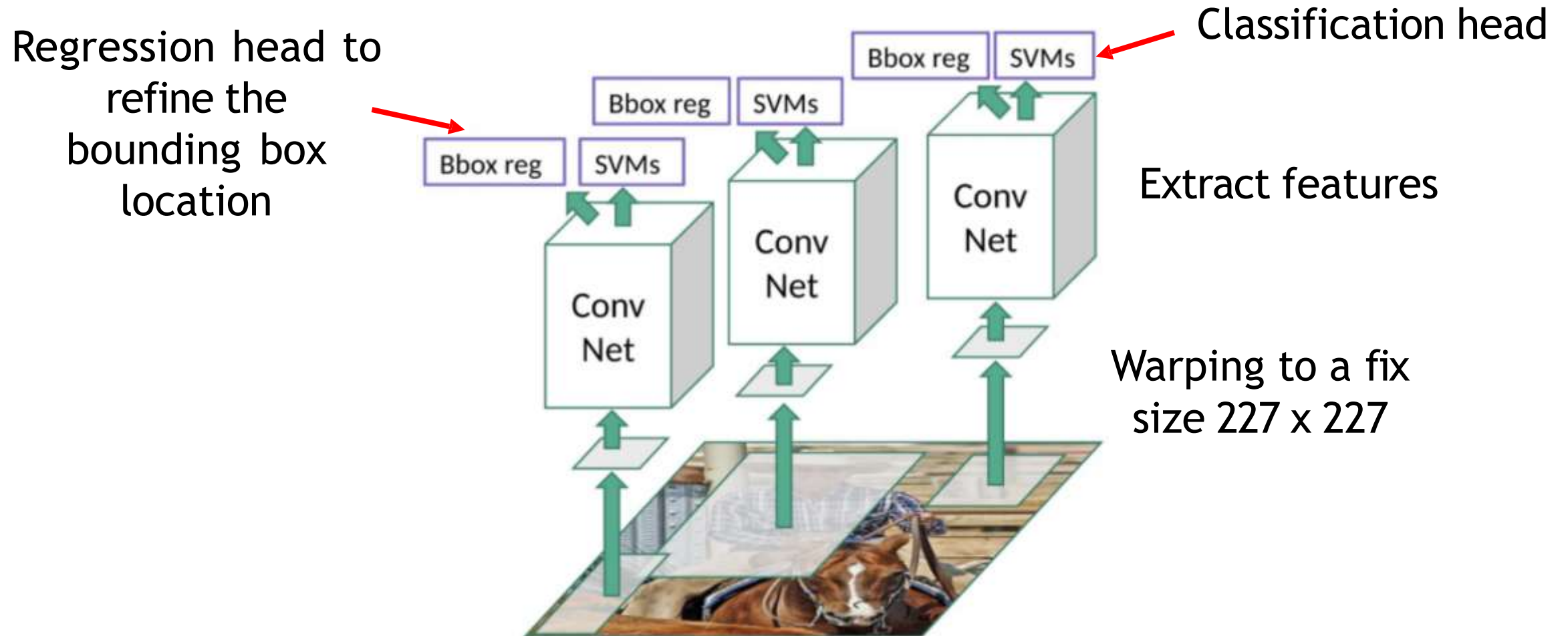


Two Stage Detectors

R-CNN



R-CNN

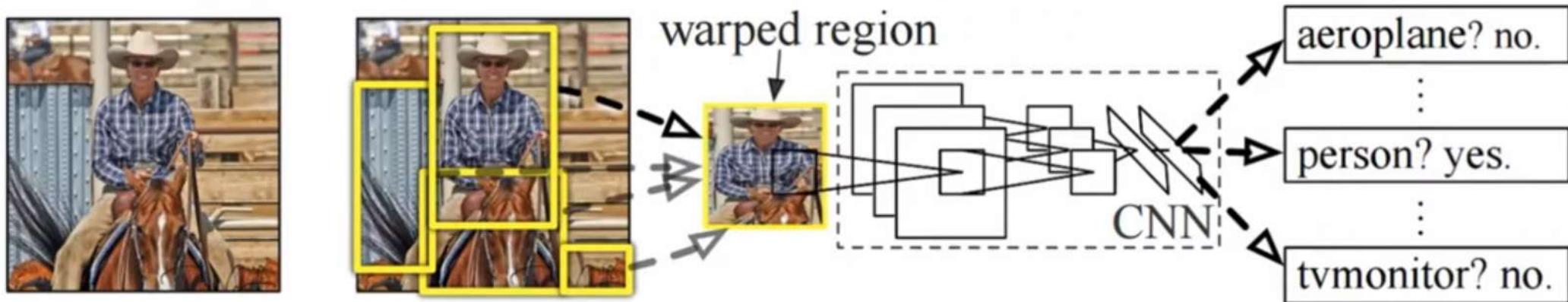


The original **R-CNN (2014)** pipeline works as follows:

- Generate ~2000 region proposals using **Selective Search**
- For each region:
 - Crop it from the original image
 - **Resize it to a fixed size (e.g., 227×227 for AlexNet)**
 - Feed it into the CNN
- Extract features
- Classify using SVM

R-CNN Limitations

R-CNN Problem #1: Slow, because it needs to run full forward pass of DNN for each region proposal.

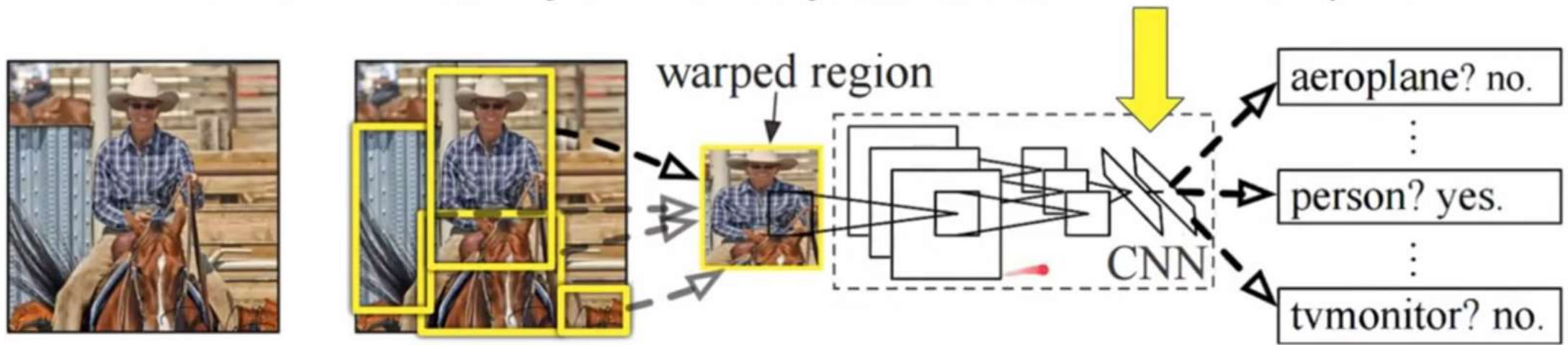


How can we avoid running multiple forward passes through the DNN ?

Solution: Feed the whole image only once through the DNN.

R-CNN Limitations

R-CNN Problem #2: The fully connected layers in the CNN force a fix input size.

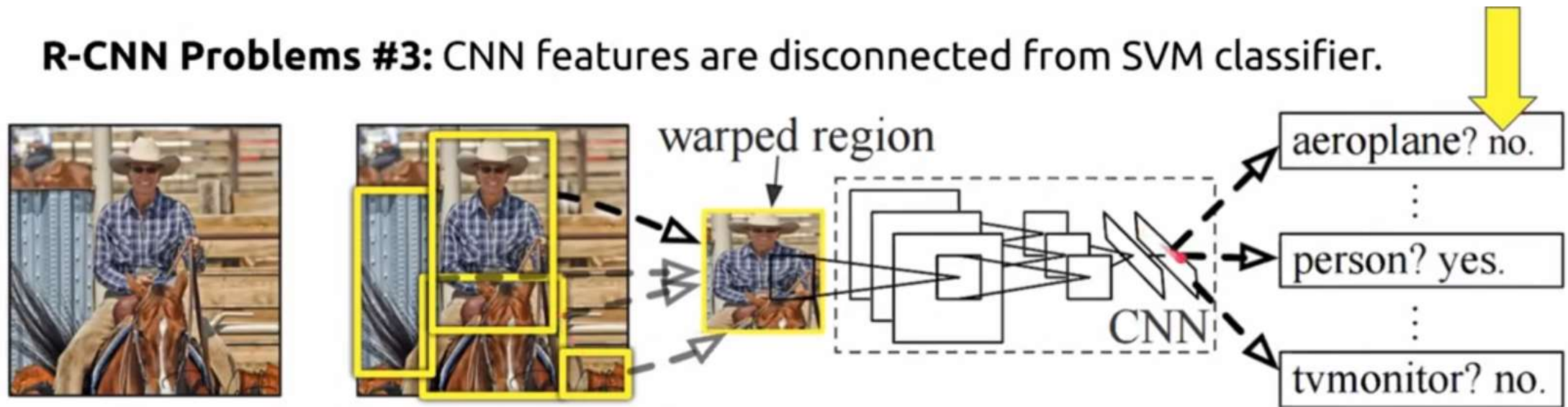


How can we force fixed-length representations in a CNN ?

(One possible) Solution: Use pooling layers.

R-CNN Limitations

R-CNN Problems #3: CNN features are disconnected from SVM classifier.



How can we learn CNN features optimized for object detection ?

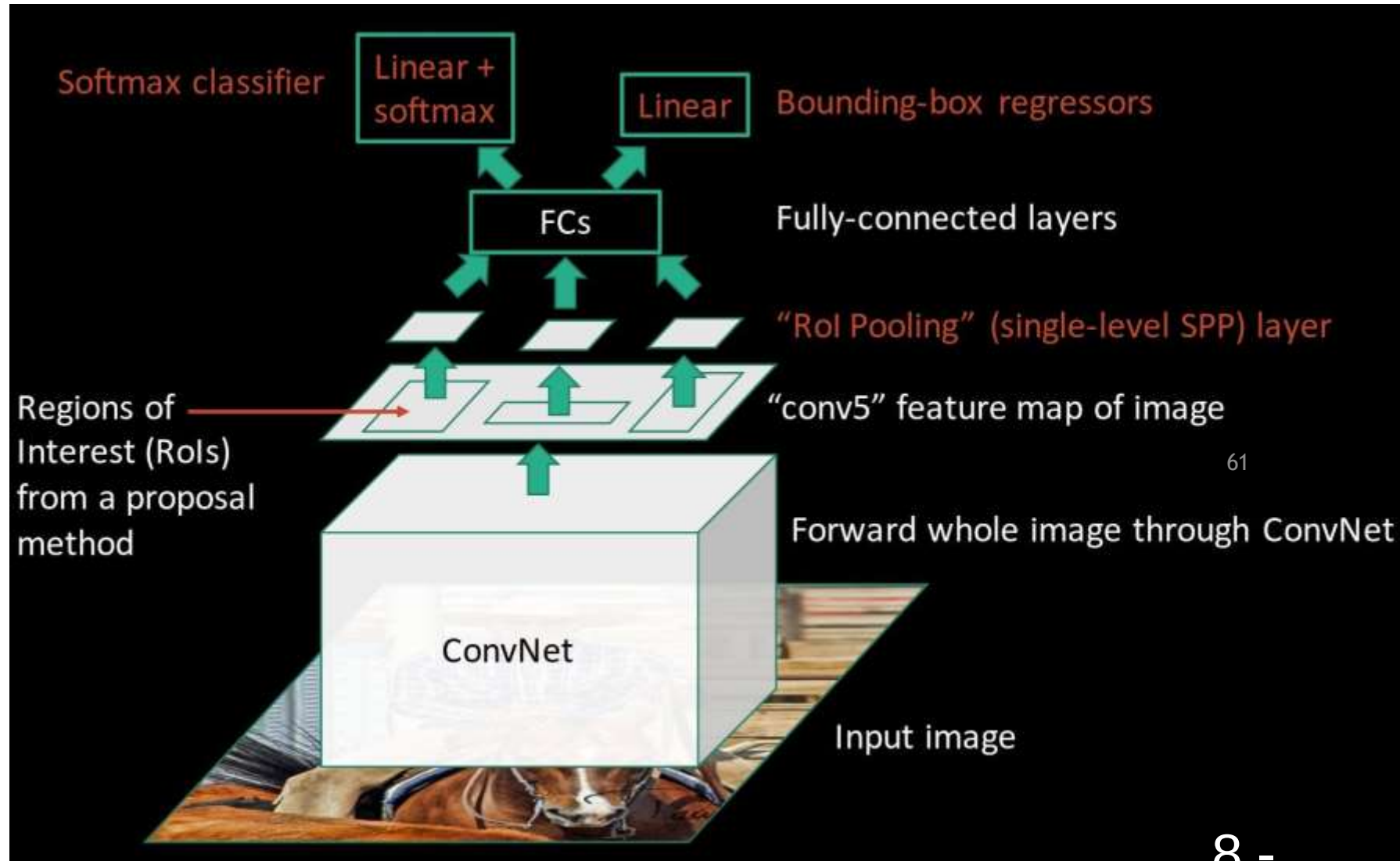
Warping can:

- Distort object shapes
- Change aspect ratio
- Hurt localization accuracy
- Lose fine spatial information
- This is one reason why R-CNN is computationally expensive and not ideal.

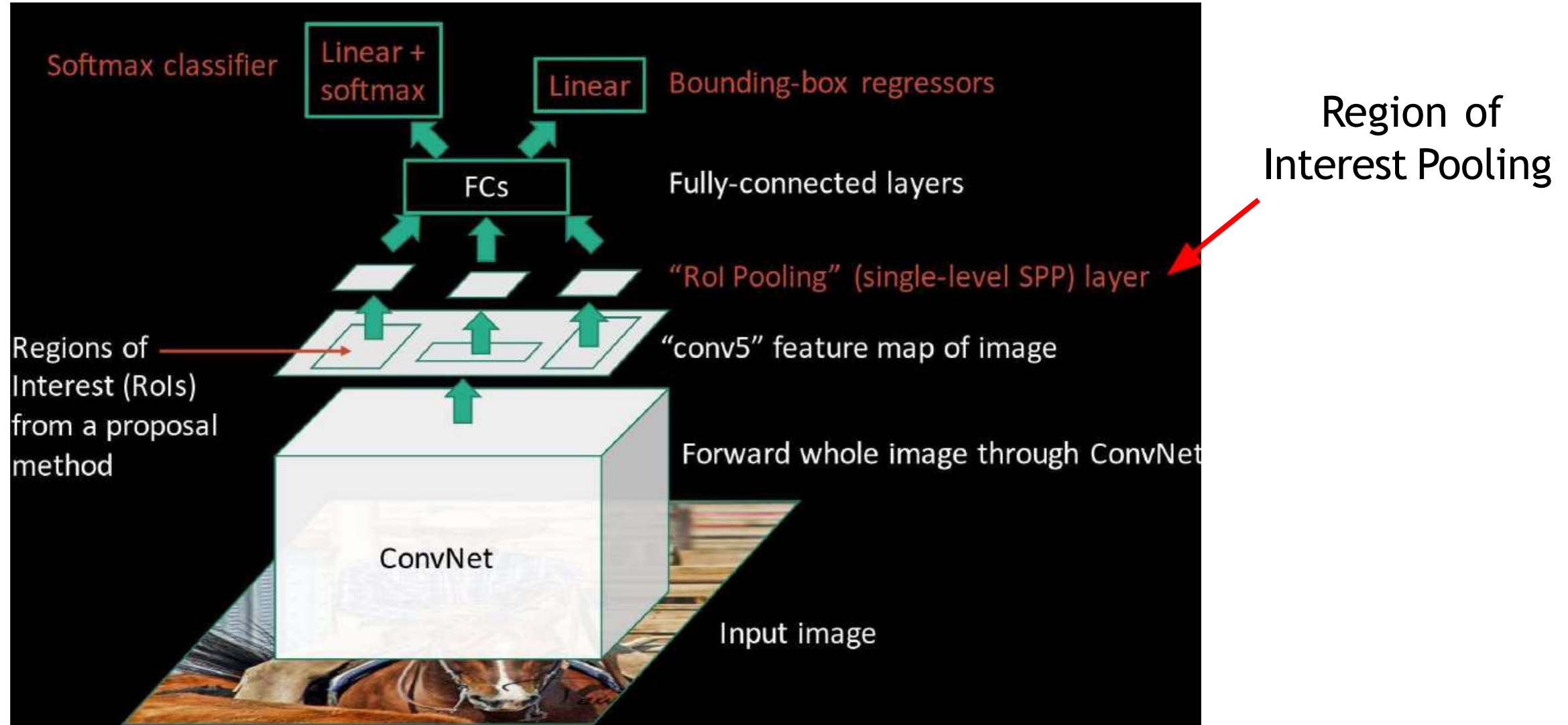
Why did they use SVM instead of softmax?

- At that time: Historical context (2013–2014)
- CNN fine-tuning for detection was not yet mature.
- Transfer learning pipelines commonly used:
- CNN as feature extractor + SVM classifier
- This was standard practice in computer vision.

Fast R-CNN



Fast R-CNN



Fast R-CNN

Input image $\rightarrow$ CNN $\rightarrow$ feature map

- Region proposals (from external method like Selective Search)
- RoI Pooling $\rightarrow$ fixed-size feature vectors
- Fully connected layers:
 - Classification (what object?)
 - Bounding box regression (where exactly?)

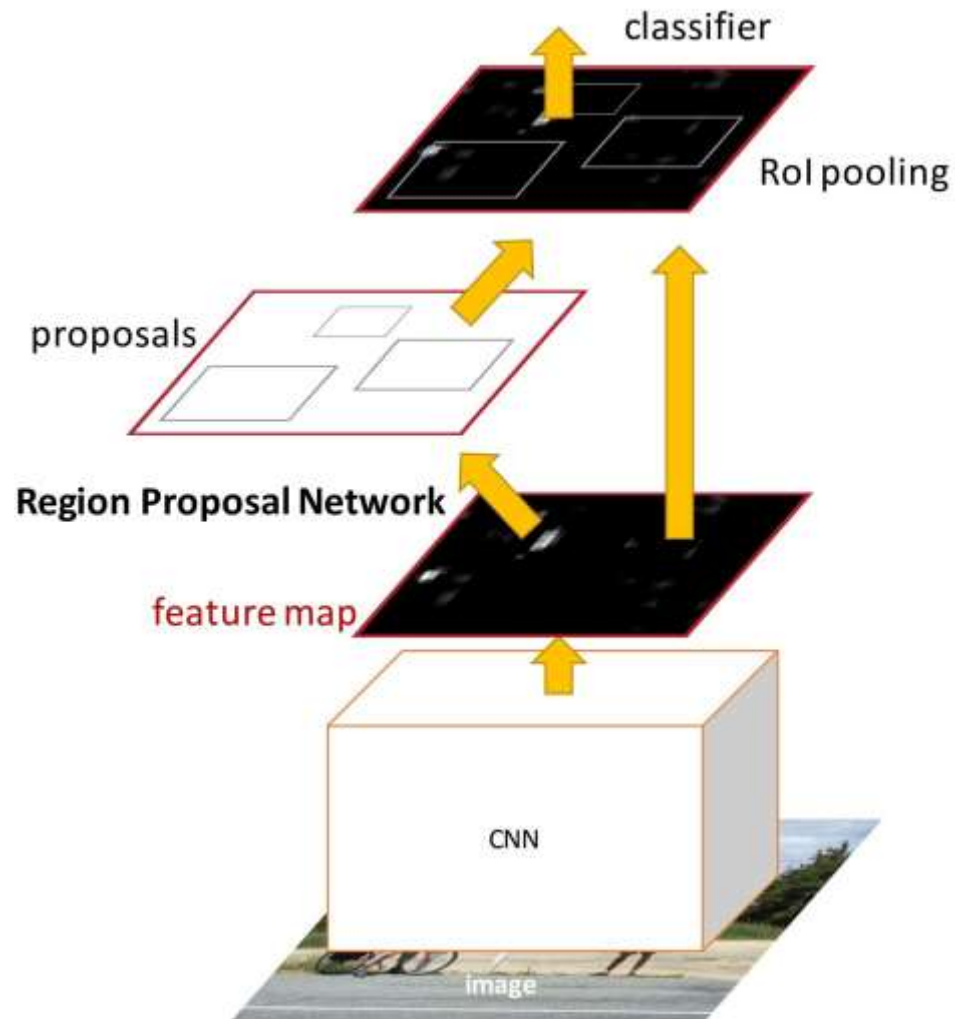
✓ **Advantages**

- Much faster than R-CNN
- Shared computation (no repeated CNN passes)
- End-to-end training (almost)

✗ **Limitation**

- Still depends on **external region proposals**, which are slow

Faster R-CNN



- Solution: Have the proposal generation integrated with the rest of the pipeline
- Region Proposal Network (RPN) trained to produce region proposals directly.
- After RPN, everything is like Fast R-CNN

Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015

Slide credit: Ross Girshick

Faster R-CNN

- **Faster R-CNN** improves Fast R-CNN by removing the slow region proposal step.
- **Key Innovation: Region Proposal Network (RPN)**
- Instead of Selective Search, Faster R-CNN introduces a **Region Proposal Network (RPN)** that:
 - Slides over the feature map
 - Generates object proposals using **anchors**
 - Predicts:
 - Objectness score (is there an object?)
 - Bounding box adjustments

Faster R-CNN

⚙️ Pipeline

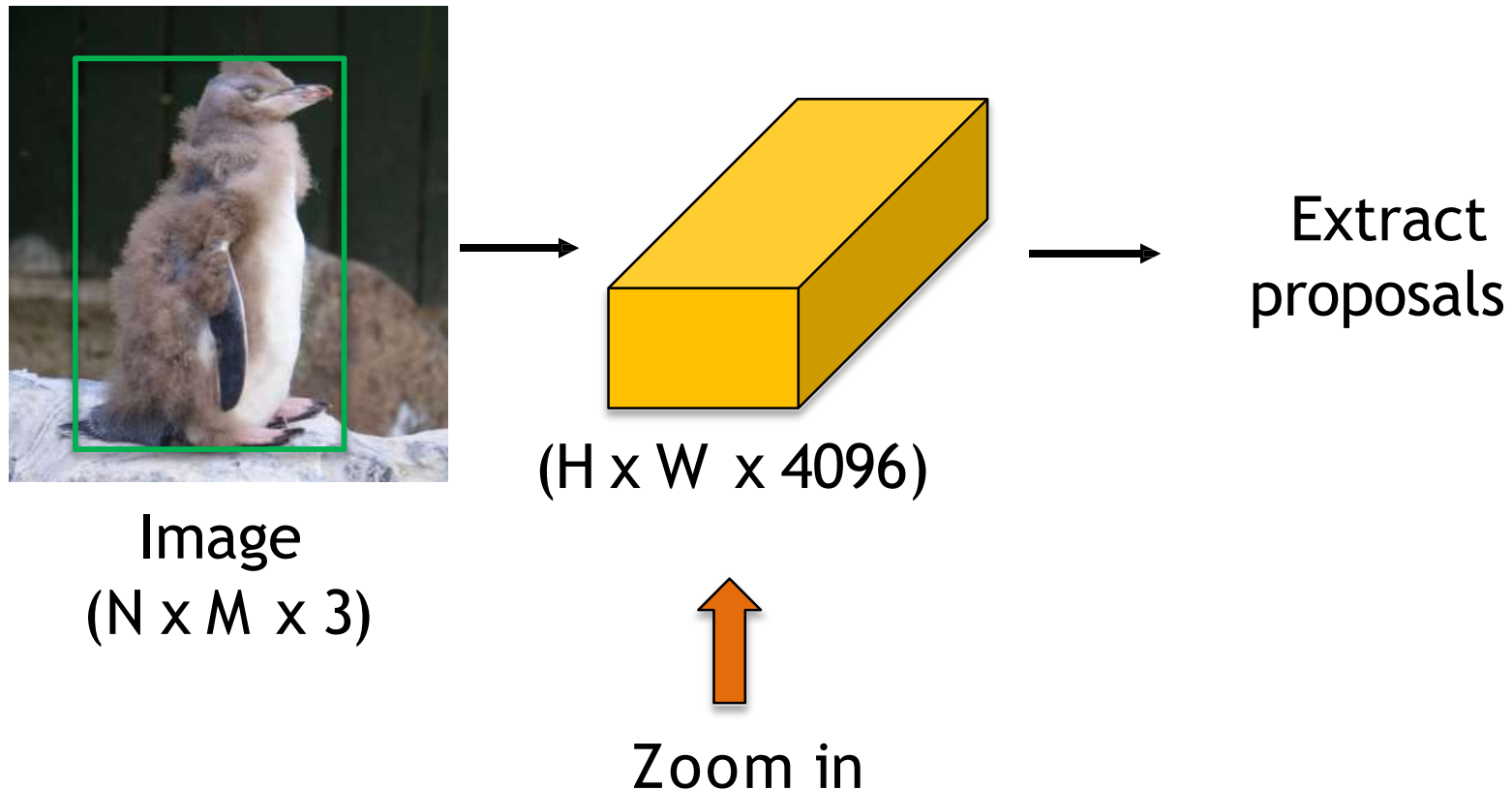
- Input image → CNN → feature map
- RPN generates region proposals
- RoI Pooling on proposals
- Classification + bounding box regression
- ☞ The key: **RPN and detector share the same CNN features**

✓ Advantages

- Much faster than Fast R-CNN
- Fully end-to-end trainable
- Better accuracy due to learned proposals

Region proposal network

- How to extract proposals



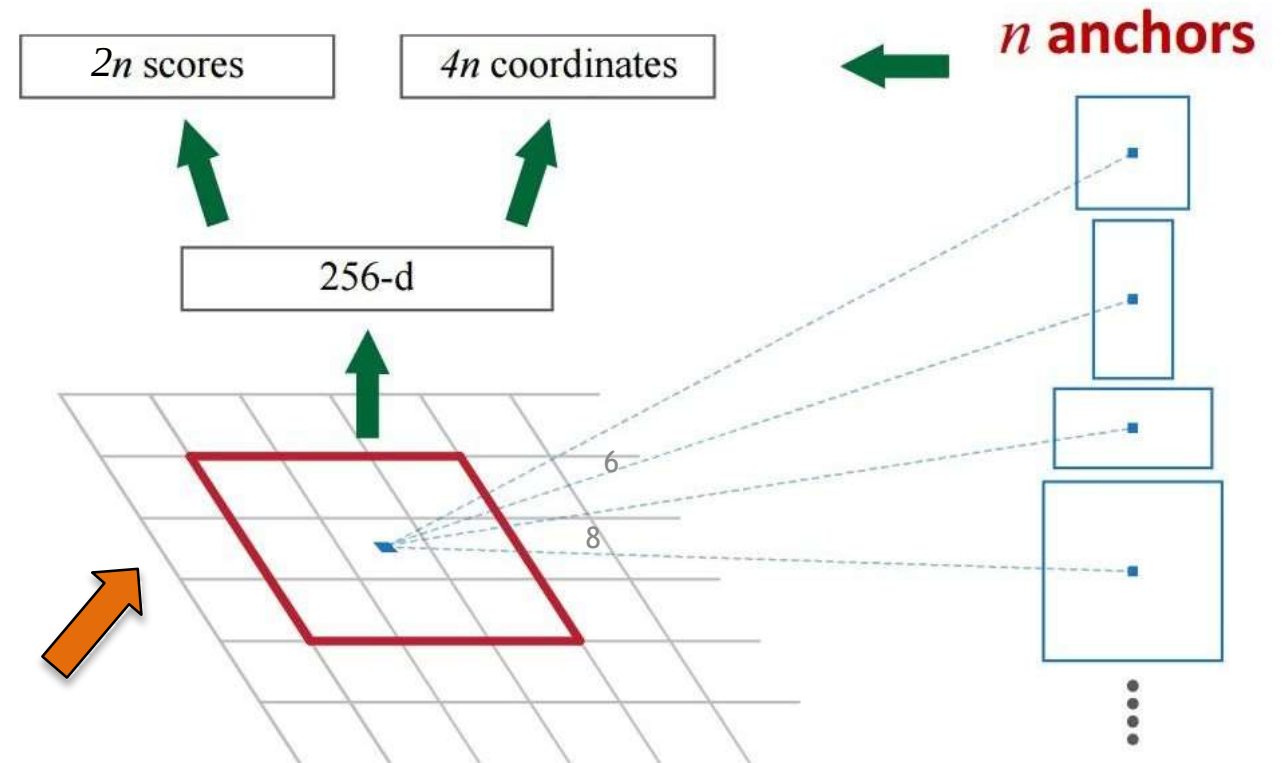
- How many proposals?
- ✓ We need to decide a fixed number

67

- Where are they placed?
- ✓ Densely

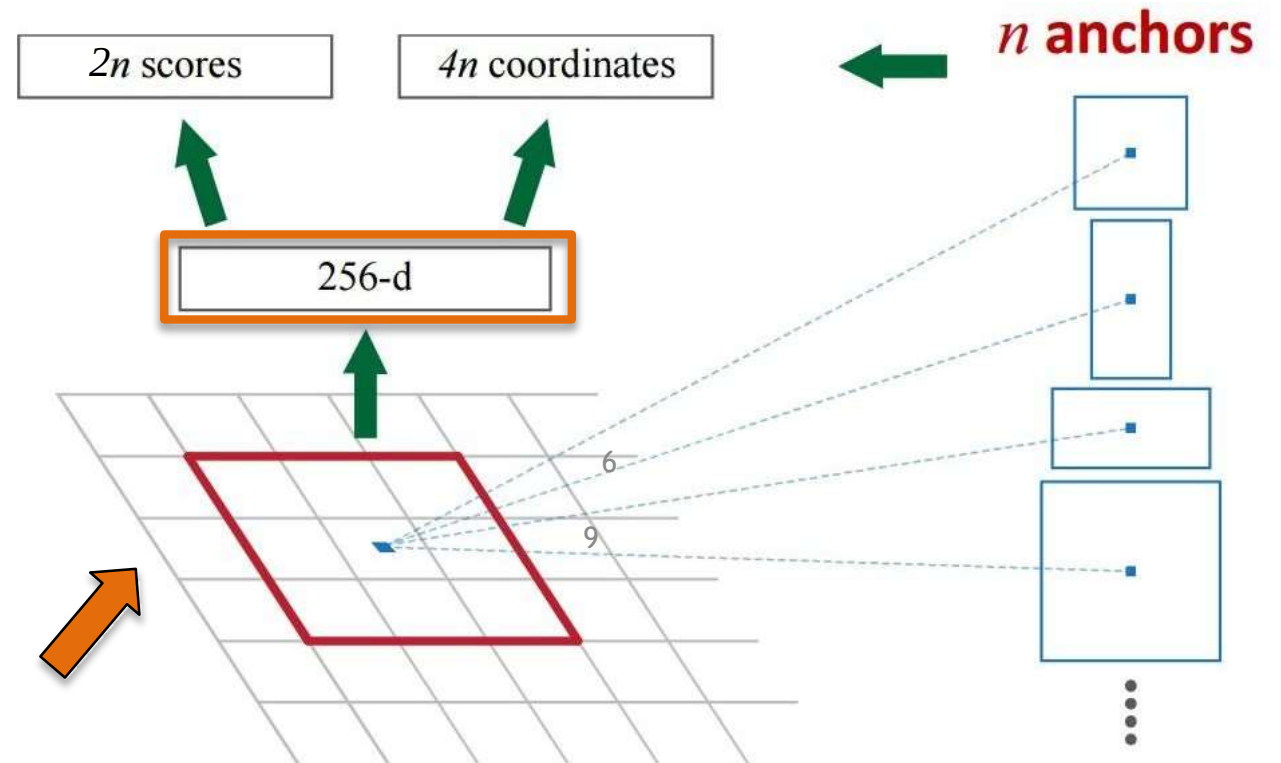
Region proposal network

- We fix the number of proposals by using a set of $n=9$ anchors *per location*.
- 9 anchors = 3 scales and 3 aspect ratios



Region proposal network

- We fix the number of proposals by using a set of $n=9$ anchors *per location*.
- 9 anchors = 3 scales and 3 aspect ratios
- We extract a descriptor *per location*



Region proposal network

- How to extract proposals

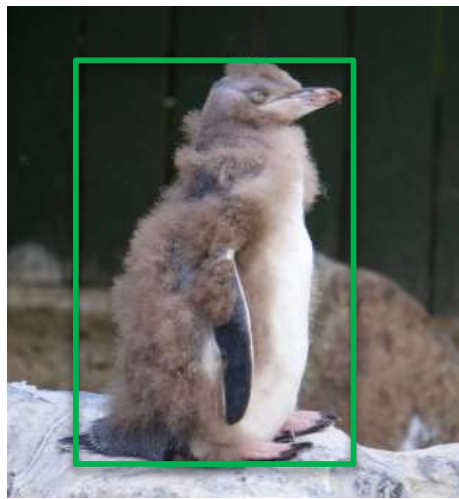
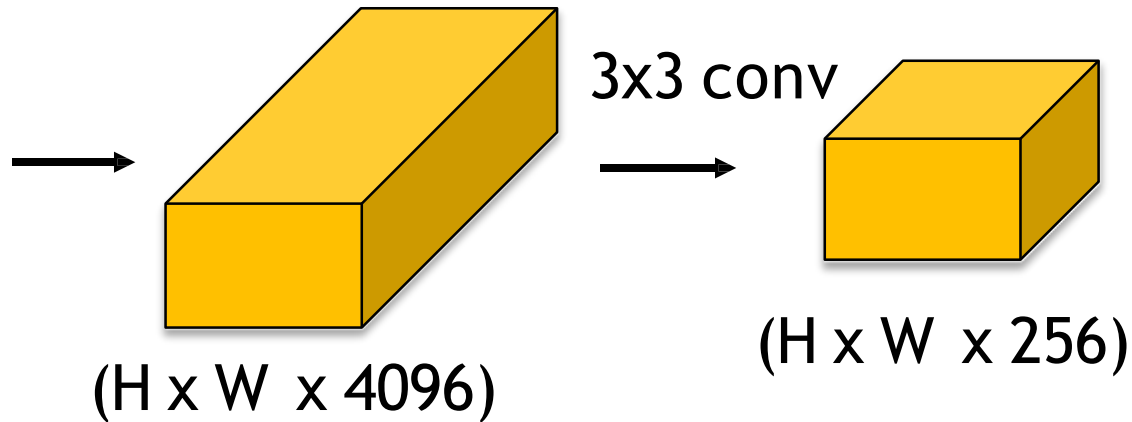


Image
($N \times M \times 3$)



70

#anchors per image? $(H \times W \times n)$

Region proposal network

- How to extract proposals

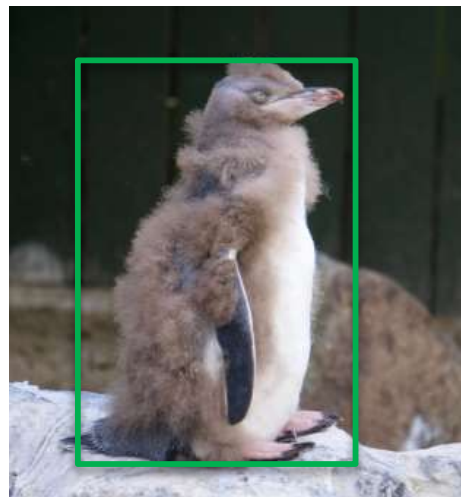
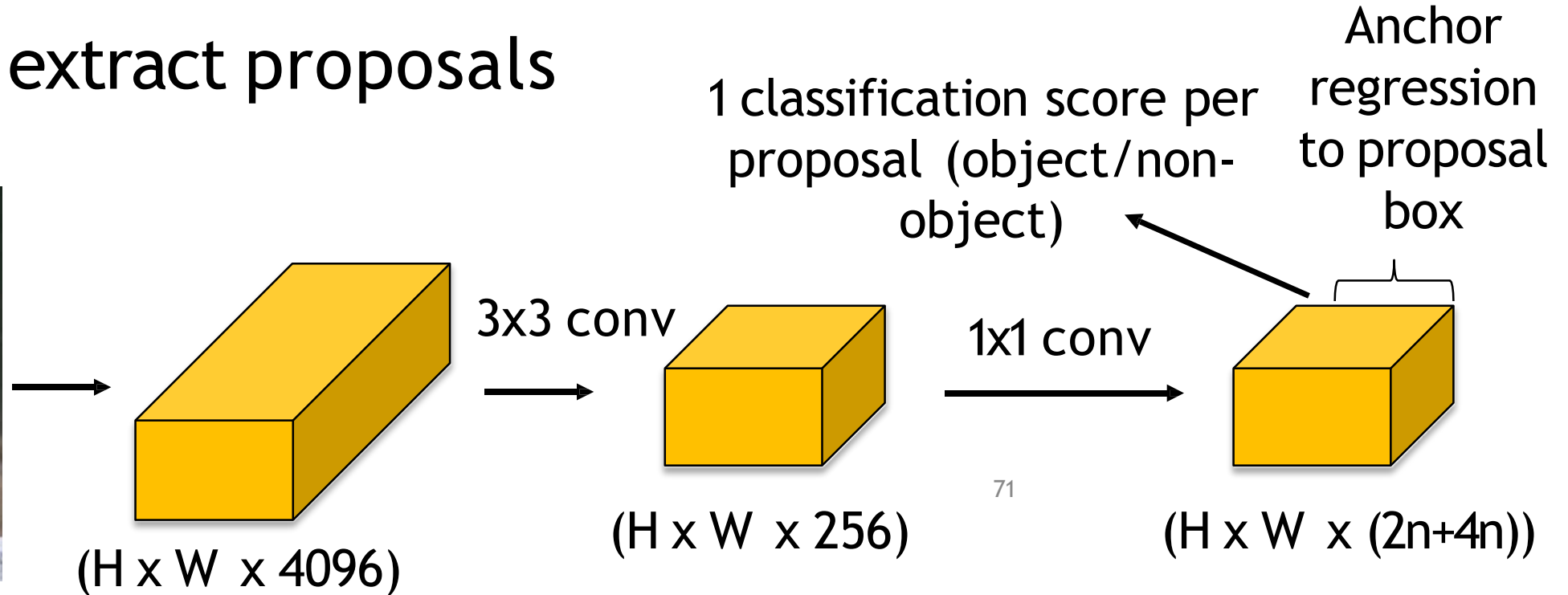
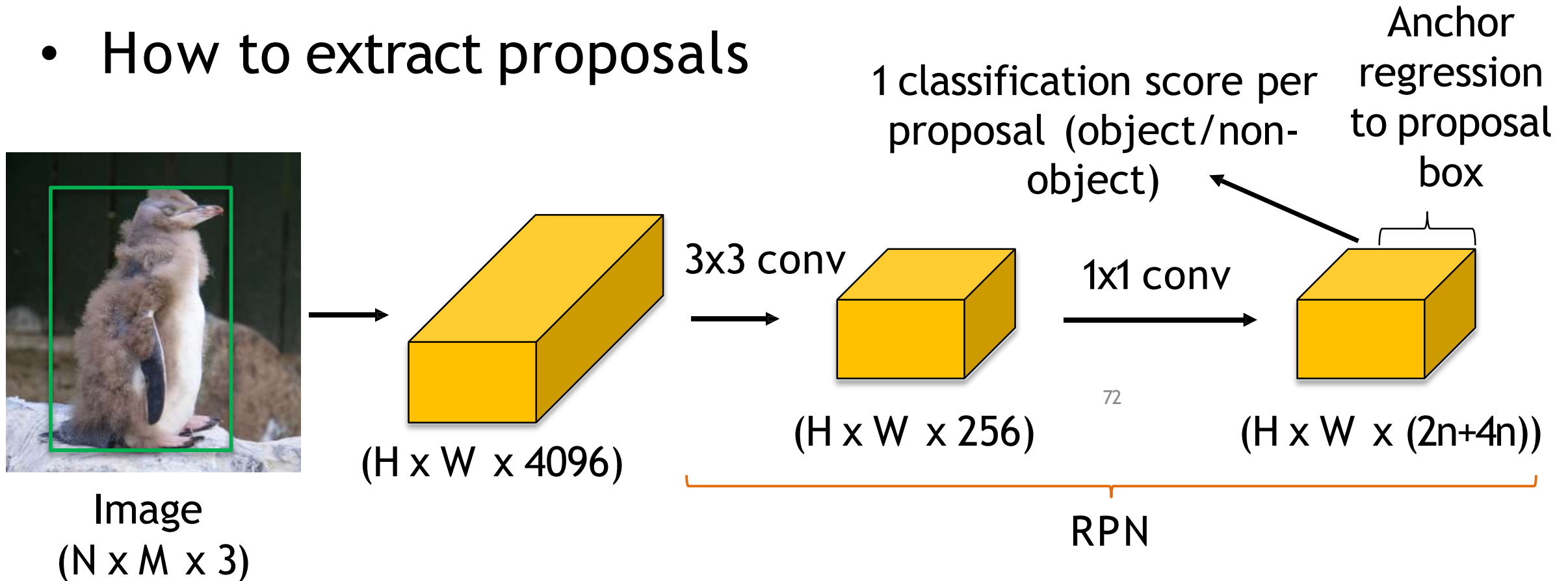


Image
($N \times M \times 3$)



Region proposal network

- How to extract proposals



Per feature map location, I get a set of anchor correction and classification into object/non-object

RPN: training and losses

- Classification ground truth: We compute p^* which indicates how much an anchor overlaps with the ground truth bounding boxes

$$p^* = 1 \quad \text{if} \quad \text{IoU} > 0.7$$

$$p^* = 0 \quad \text{if} \quad \text{IoU} < 0.3$$

73

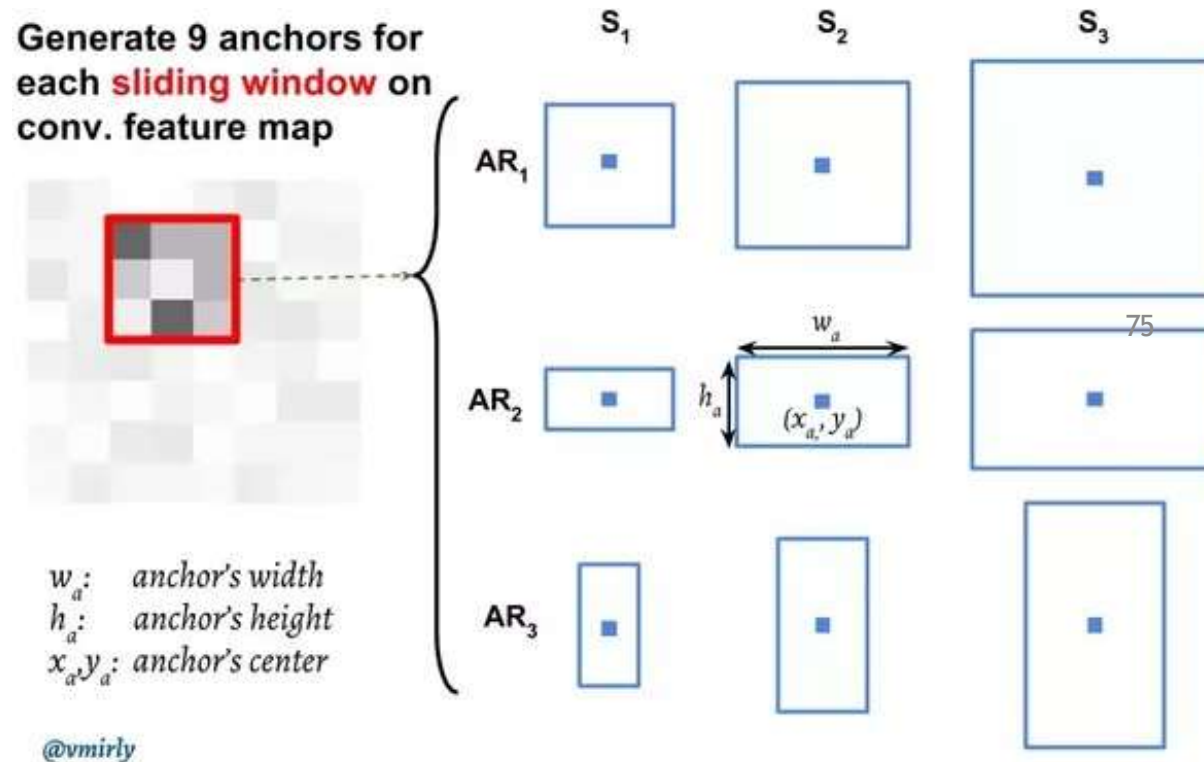
- 1 indicates the anchor represent an object (foreground) and 0 indicates background object. The rest do not contribute to the training.

RPN: training and losses

- For an image, we randomly sample 256 anchors to form a mini-batch (balanced objects vs. non-objects)
- We calculate the classification loss (binary cross-entropy).
- Those anchors that do contain an object are used to compute the regression loss

RPN: training and losses

- Each anchor is described by the center position, width and height x_a, y_a, w_a, h_a



RPN: training and losses

- Each anchor is described by the center position, width and height x_a, y_a, w_a, h_a
- What the network actually predicts are t_x, t_y, t_w, t_h

Normalized x $t_x = (x - x_a)/w_a,$ $t_y = (y - y_a)/h_a,$ Normalized y

Normalized width $t_w = \log(w/w_a),$ $t_h = \log(h/h_a),$ Normalized height

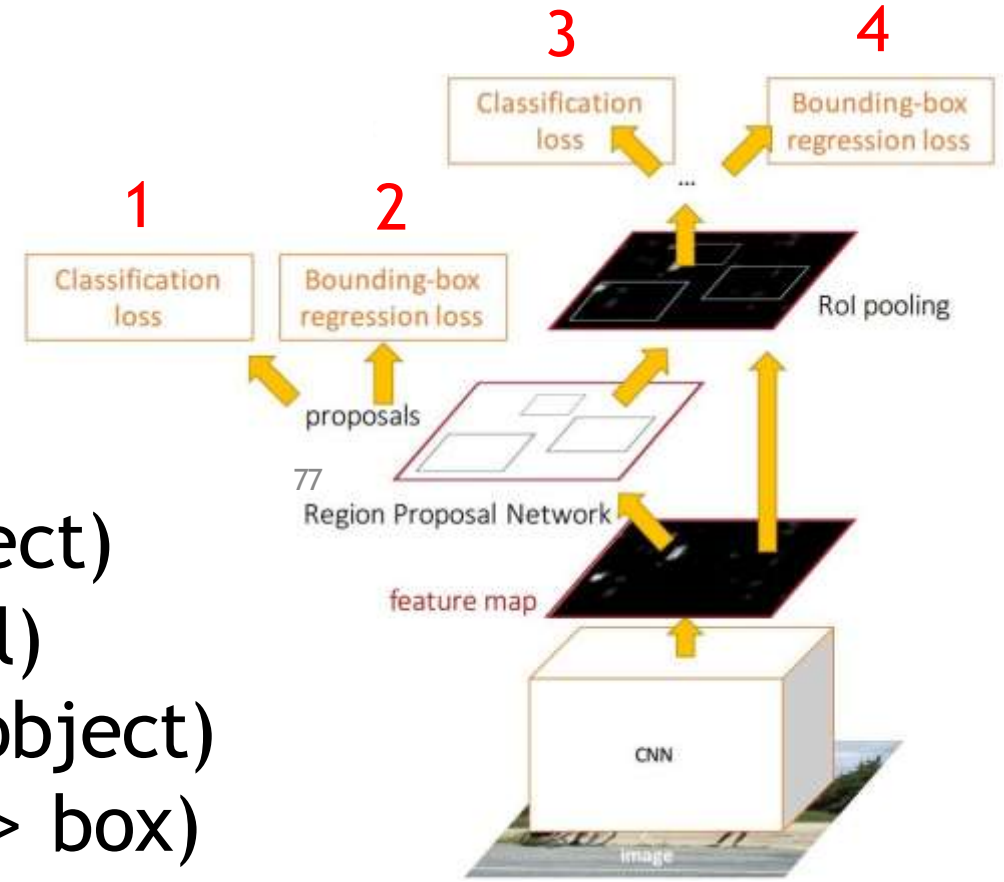
- Smooth L1 loss on regression targets

Faster R-CNN: Training

- First implementation, training of RPN separate from the rest.
- Now we can train jointly!

- Four losses:

1. RPN classification (object/non-object)
2. RPN regression (anchor -> proposal)
3. Fast R-CNN classification (type of object)
4. Fast R-CNN regression (proposal -> box)



Faster R-CNN

- 10x faster at test time wrt Fast R-CNN
- Trained end-to-end including feature extraction, region proposals, classifier and regressor
- More accurate, since proposals are learned. RPN is fully convolutional

Faster R-CNN: Results

	R-CNN	Fast R-CNN	Faster R-CNN
Test time per image (with proposals)	50 seconds	2 seconds	0.2 seconds
(Speedup)	1x	25x	250x
mAP (VOC 2007)	66.0	66.9	66.9

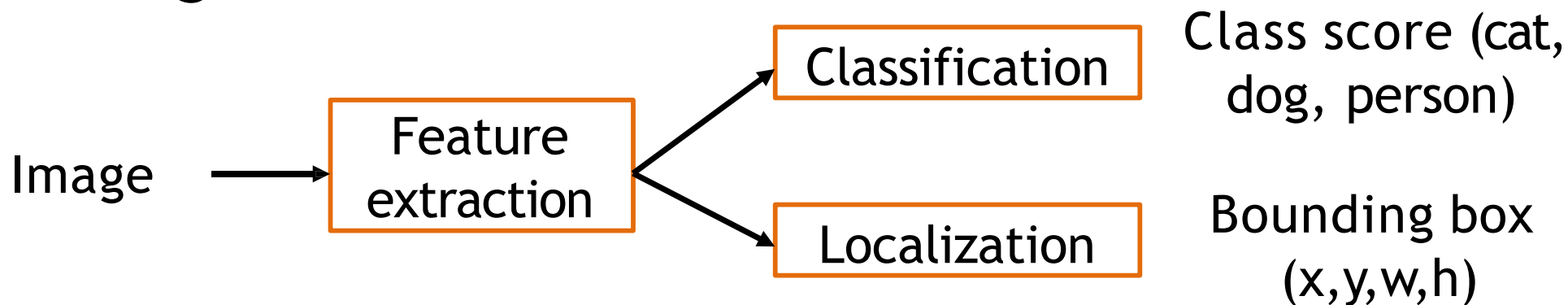
Related works

- Shrivastava, Gupta, Girshick. “Training region-based object detectors with online hard example mining”. CVPR 2016.
- Dai, Li, He and Sun. “R-FCN: Object detection via region-based fully convolutional networks”. 2016.
- Dai, Qi, Xiong, Li, Zhang, Hu and Wei. “Deformable convolutional networks”. ICCV 2017.
- Lin, Dollar, Girshick, He, Hariharan and Belongie. “**Feature Pyramid Networks for object detection**”. CVPR 2017.

One Stage Detectors

Types of object detectors

- One-stage detectors



- Two-stage detectors



YOLO: You Only Look Once

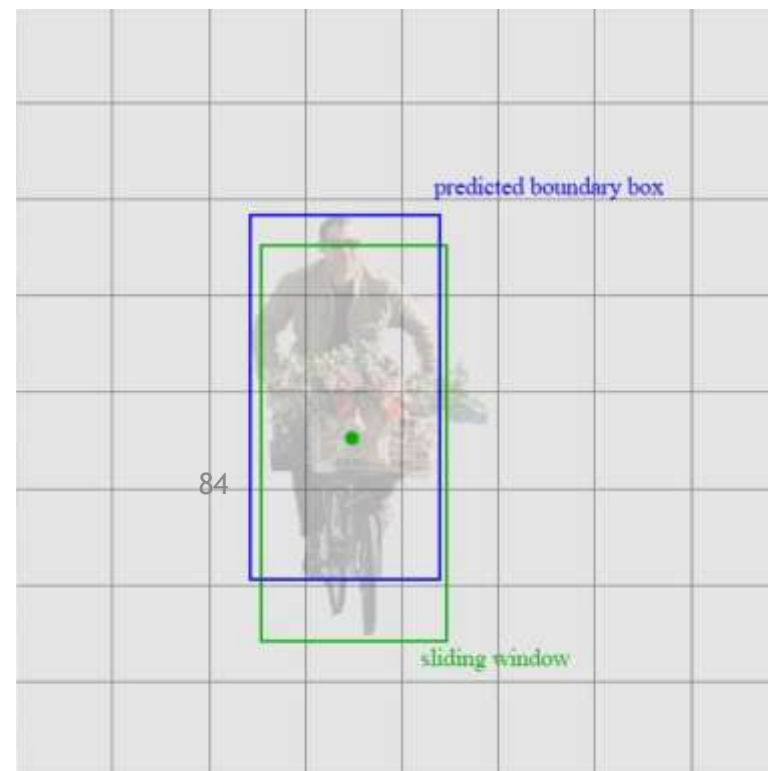
- Recall sliding window object detection
- To make it efficient, we will “slide our window” only on certain locations of the image
- We divide our image in a grid.



Redmon et al, “You only look once: Unified real-time object detection”, CVPR 2016.

YOLO: You Only Look Once

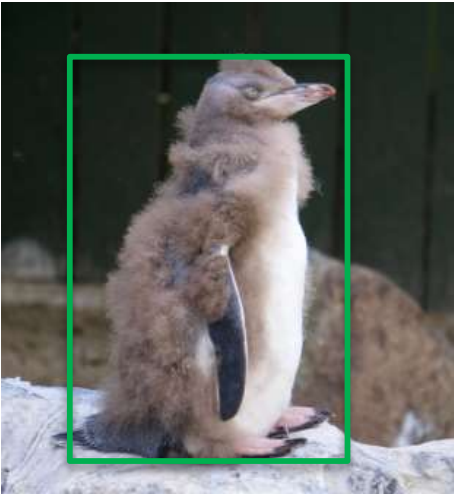
- We will place a box at the center of each cell in the grid, and this will be our initial box guess for that object



Redmon et al, "You only look once: Unified real-time object detection", CVPR 2016.

YOLO: You Only Look Once

- Direct regression from image to box coordinates



Convolutional neural network

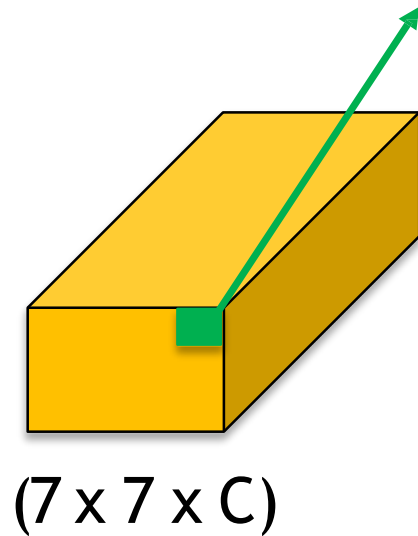
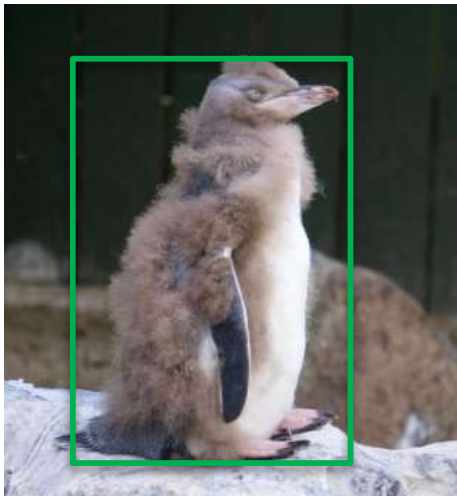


Detections

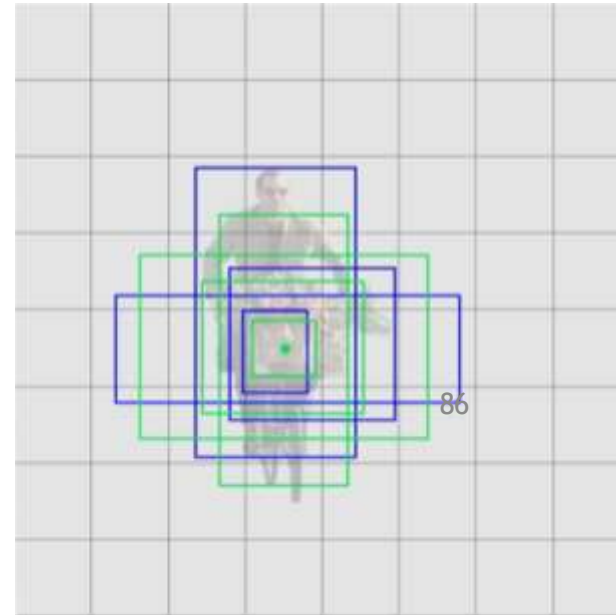
YOLO: You Only Look Once

- In YOLOv2 we use anchor boxes

For each grid location we predict n boxes



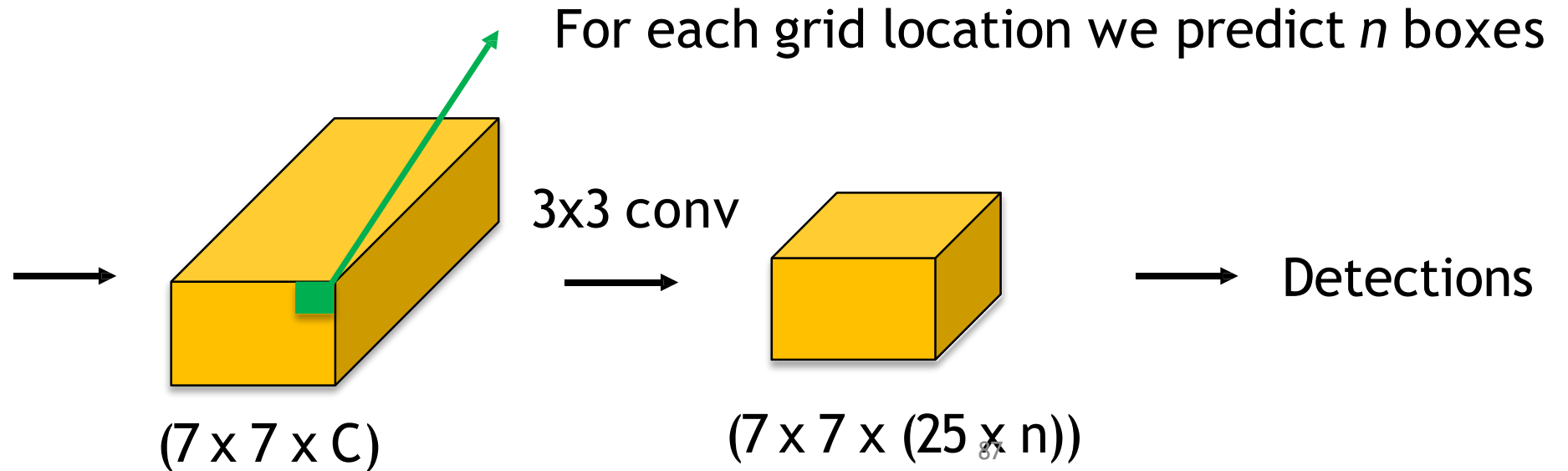
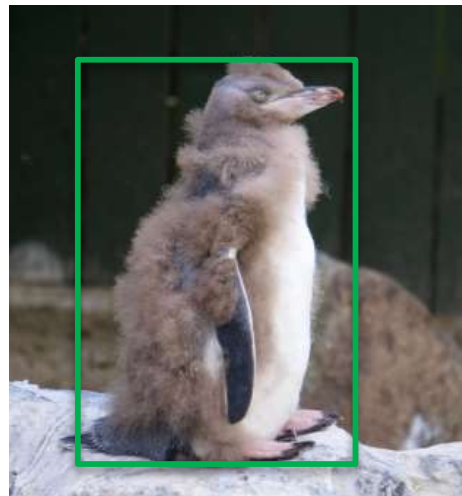
$(7 \times 7 \times C)$



Green = anchors
Blue = predictions

YOLO: You Only Look Once

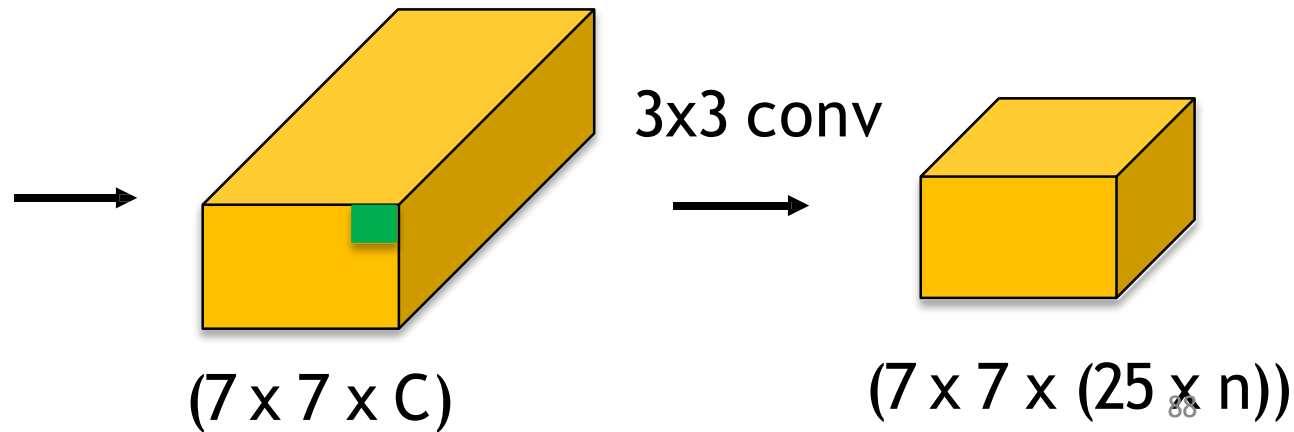
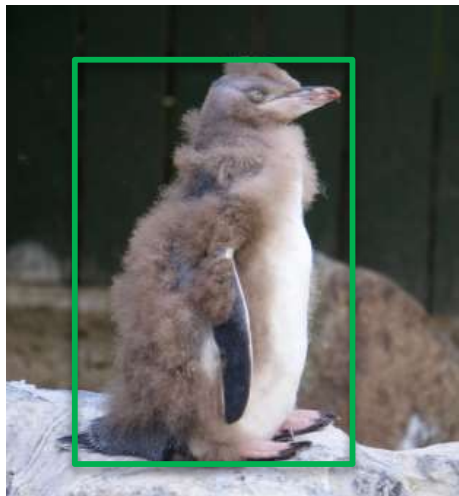
- In YOLOv2 we use anchor boxes



Similar to Faster R-CNN region proposal network

YOLO: You Only Look Once

- In YOLOv2 we use anchor boxes

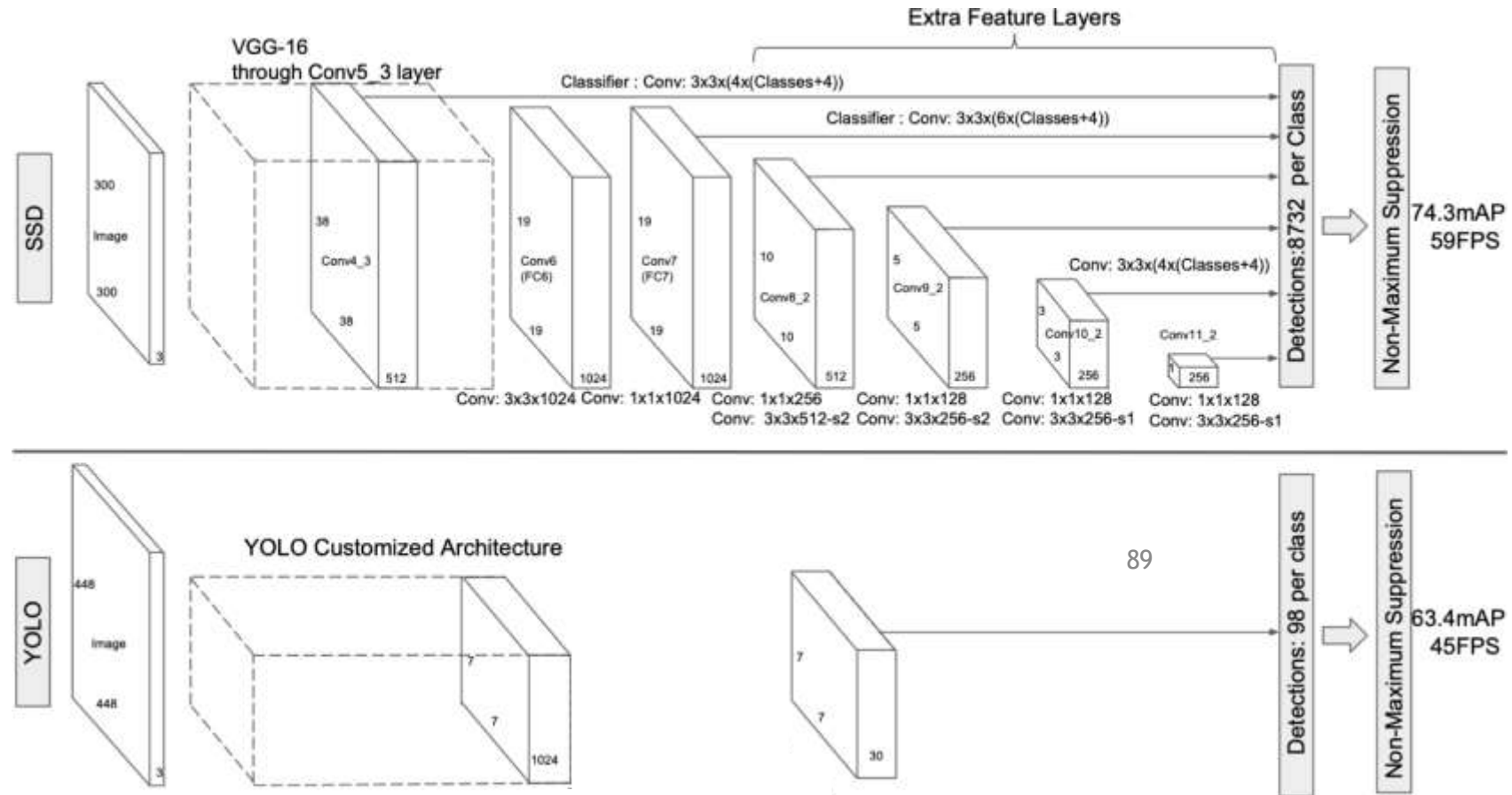


- 1 class prediction (20)
- Anchor relative box regression (4)
- Object/non-object (1)

SSD: Single Shot multibox Detector

- SSD predicts at different scales

Only few extra layers!



- PROS: YOLO and SSD
 - Very fast
 - End-to-end trainable and fully convolutional
 - SSD detects more objects than YOLO
- CONS:
 - Performance is not as good as two-stage⁹⁰ detectors
 - Difficulty with small objects

Problem with one-stage detectors

- Two-stage detectors:
 - Classification only work on “interesting” foreground regions (proposals, ~1-2k). Most background examples are already filtered out.
 - Class balance between foreground and background objects is manageable.
 - Classifier can concentrate on analyzing proposals with rich information content

Problem with one-stage detector

- One-stage detectors:
 - Many locations need to be analyzed (100k)
densely covering the image → foreground-background imbalance
 - Hard negative mining is useful, but not sufficient

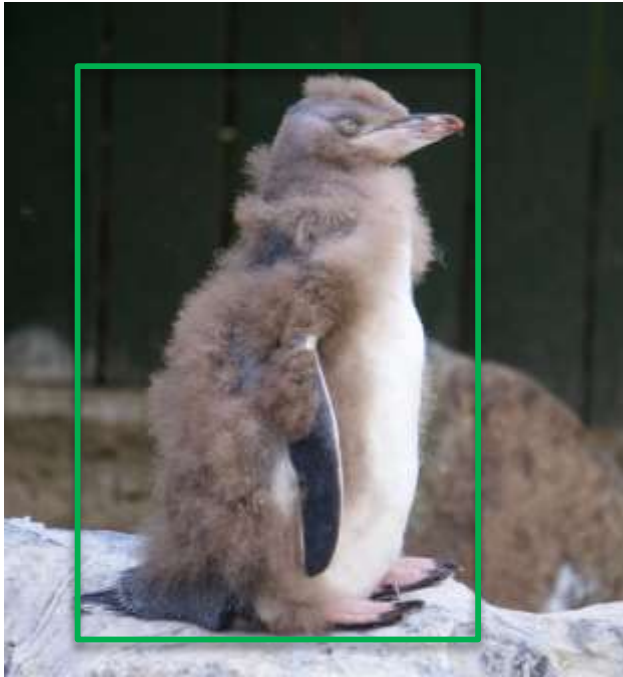


Many negative examples, no useful signal

Few positive examples, rich information for learning

Detection Evaluation

Detection Evaluation



FP = False positives. FN = False negatives

Detection Evaluation

- Precision: how accurate your predictions are.

$$Precision = \frac{TP}{TP + FP}$$

TP → Good boxes detected
TP + FP → All boxes that you detected

- Recall: how good you are at finding all positives

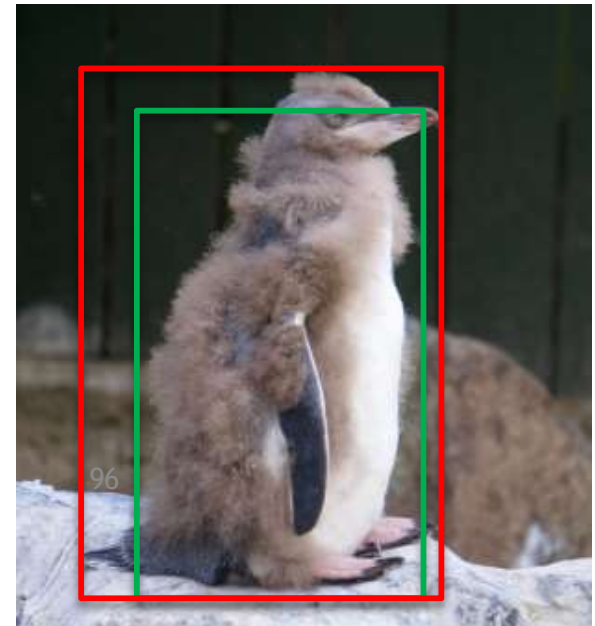
$$Recall = \frac{TP}{TP + FN}$$

TP → Good boxes detected
TP + FN → All ground truth boxes

FP = False positives. FN = False negatives

Detection evaluation

- What is a positive?
- We use the Intersection over Union (IoU) or Jaccard Index
- If $\text{IoU} > 0.5 \rightarrow$ positive match
- Depending on the dataset it is 0.5 or 0.7



FP = False positives. FN = False negatives

Detection evaluation: AP

- Mean Average Precision (mAP)
 - For each image and class independently, rank the predicted boxes by confidence score. Assign the boxes to the corresponding ground truth if $\text{IoU} > 0.5$.
 - Each ground truth box can only be matched to one predicted box.
 - For each class independently, compute Average Precision
 - Mean over all classes to obtain mAP
 - mAP measures object detection performance by averaging precision over recall for each class and then taking the mean across all

Average Precision

- Rank predictions by confidence

Rank	True/false	Precision	Recall
1	T		
2	T		
3	F		
4	T		
5	F		
6	F		
7	T		
8	F		
9	T		
10	T		
11	F		
12	F		

My method predicts 12
boxes, 6 of them are true
boxes present in the
image

98

Average Precision

- Rank predictions by confidence

Rank	True/false	Precision	Recall
1	T	1.0	0.17
2	T		
3	F		
4	T		
5	F		
6	F		
7	T		
8	F		
9	T		
10	T		
11	F		
12	F		

Average Precision

- Rank predictions by confidence

Rank	True/false	Precision	Recall
1	T	1.0	0.17
2	T	1.0	0.33
3	F		
4	T		
5	F		
6	F		
7	T		
8	F		
9	T		
10	T		
11	F		
12	F		

10
0

Average Precision

- Rank predictions by confidence

Rank	True/false	Precision	Recall
1	T	1.0	0.17
2	T	1.0	0.33
3	F	0.67	0.33
4	T		
5	F		
6	F		
7	T		
8	F		
9	T		
10	T		
11	F		
12	F		

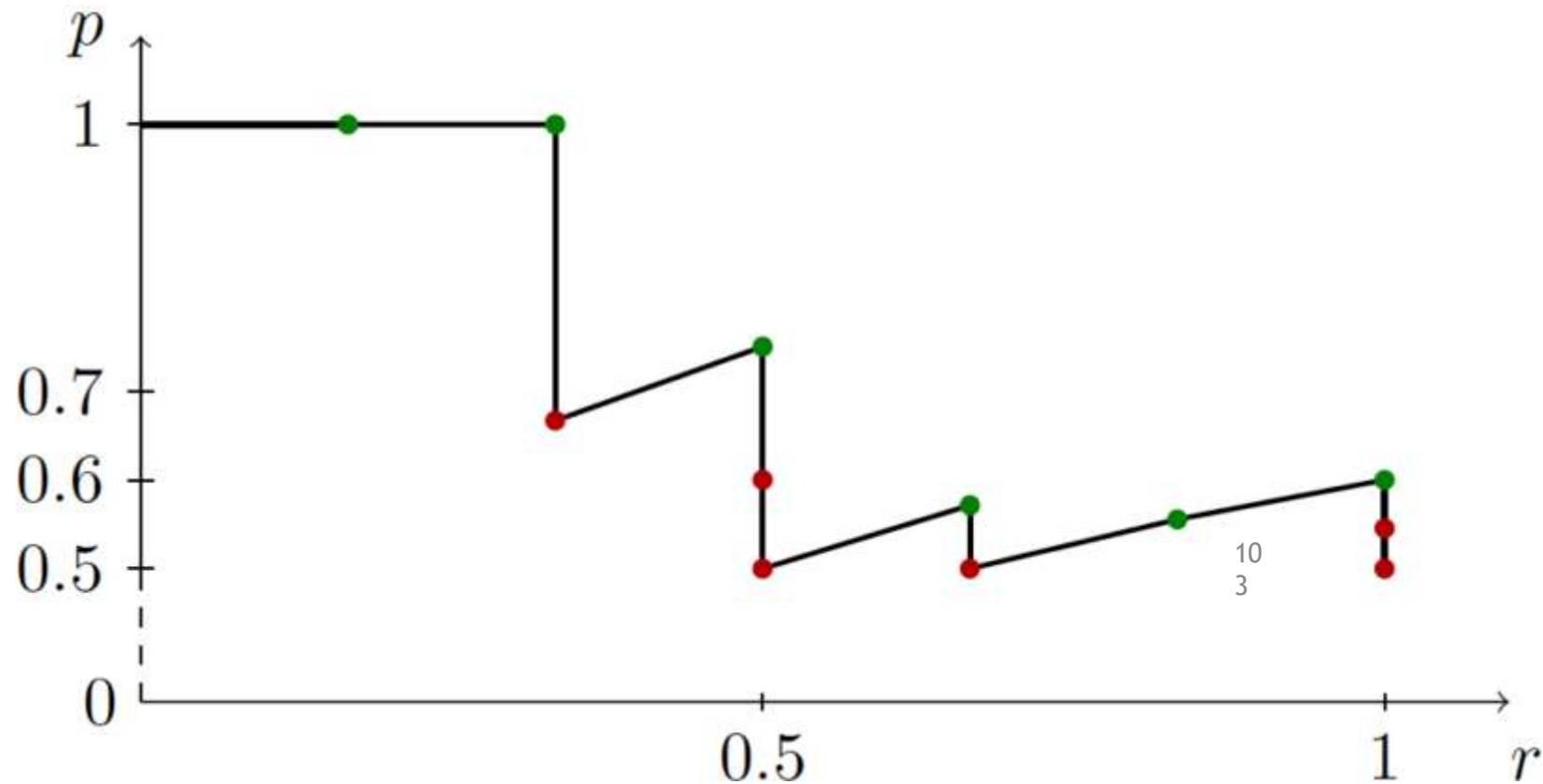
Average Precision

- Rank predictions by confidence

Rank	True/false	Precision	Recall
1	T	1.0	0.17
2	T	1.0	0.33
3	F	0.67	0.33
4	T	0.75	0.5
5	F	0.6	0.5
6	F	0.5	0.5
7	T	0.57	0.67
8	F	0.5	0.67
9	T	0.56	¹⁰ ₂ 0.83
10	T	0.6	1.0
11	F	0.55	1.0
12	F	0.5	1.0

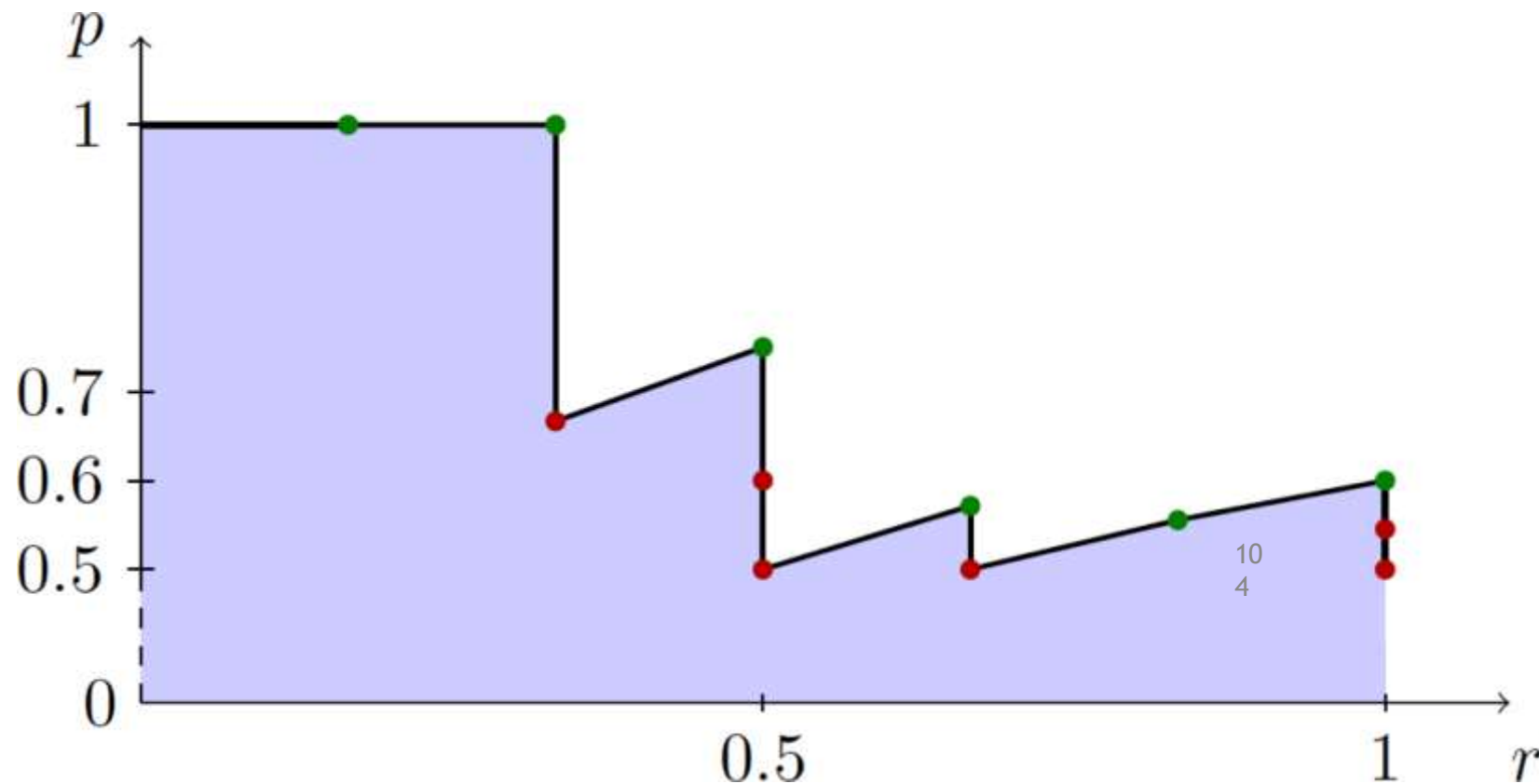
Average Precision

- Plot Precision vs Recall



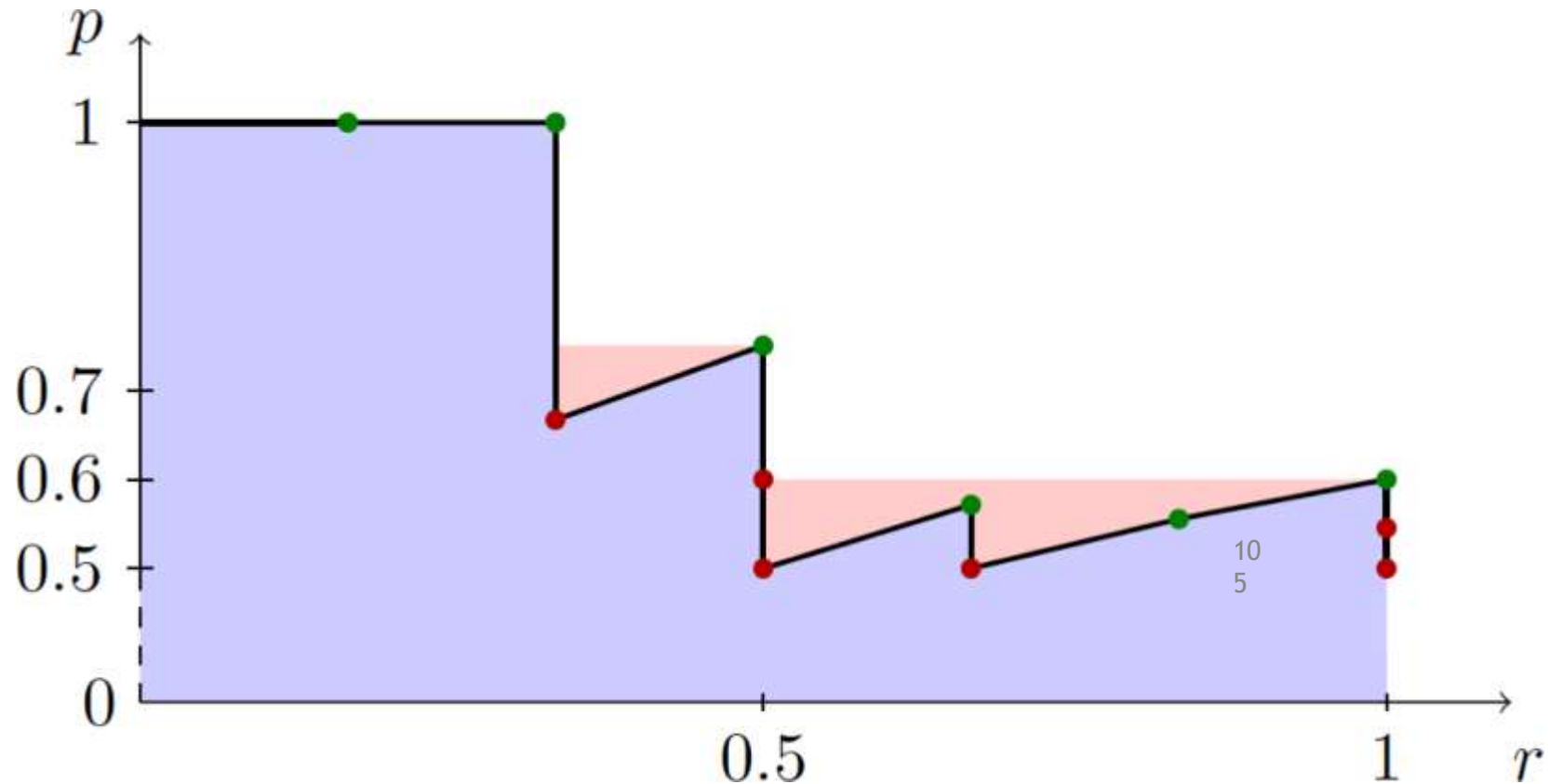
Average Precision

- Average precision = area under curve



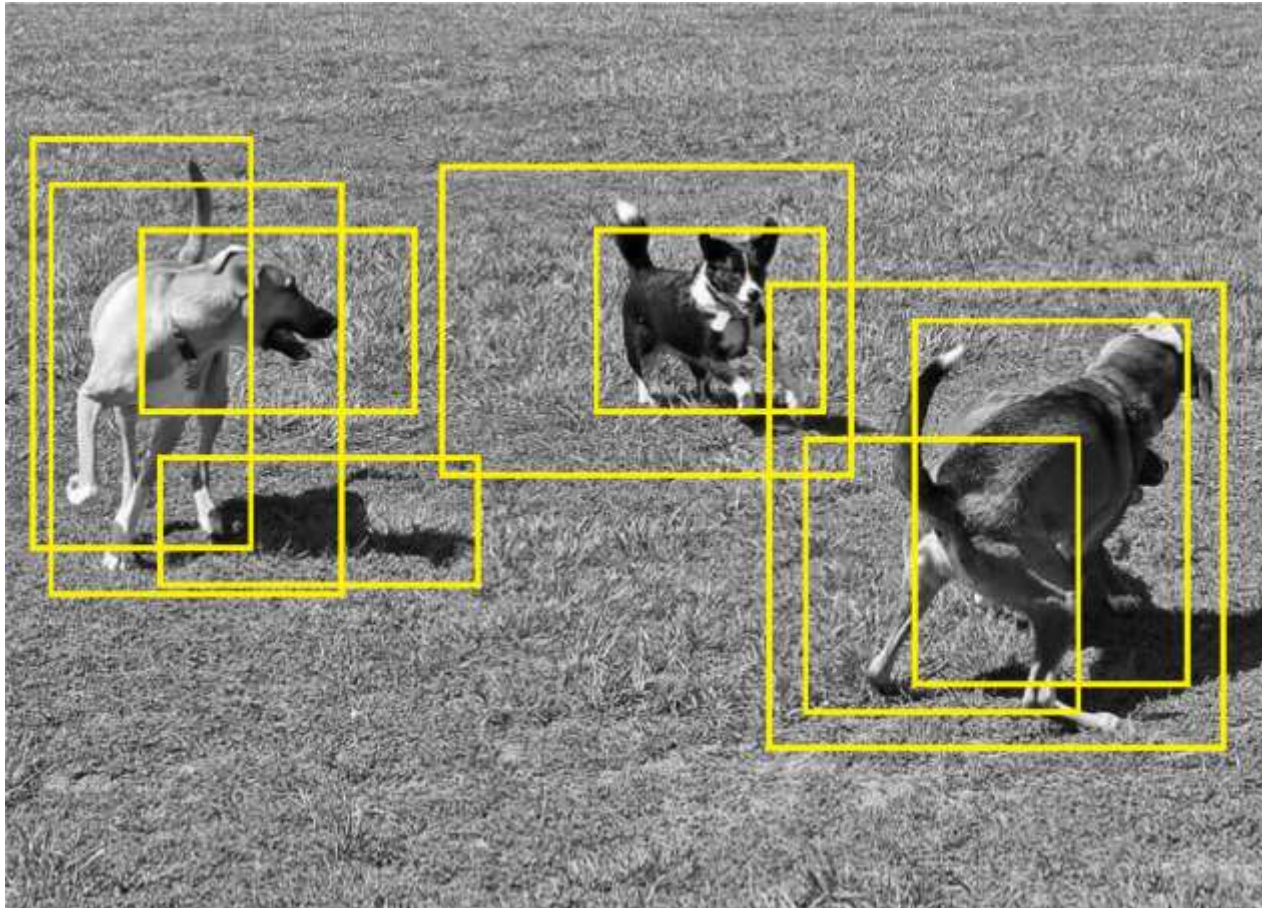
Average Precision

- Average precision = area under curve (filled)



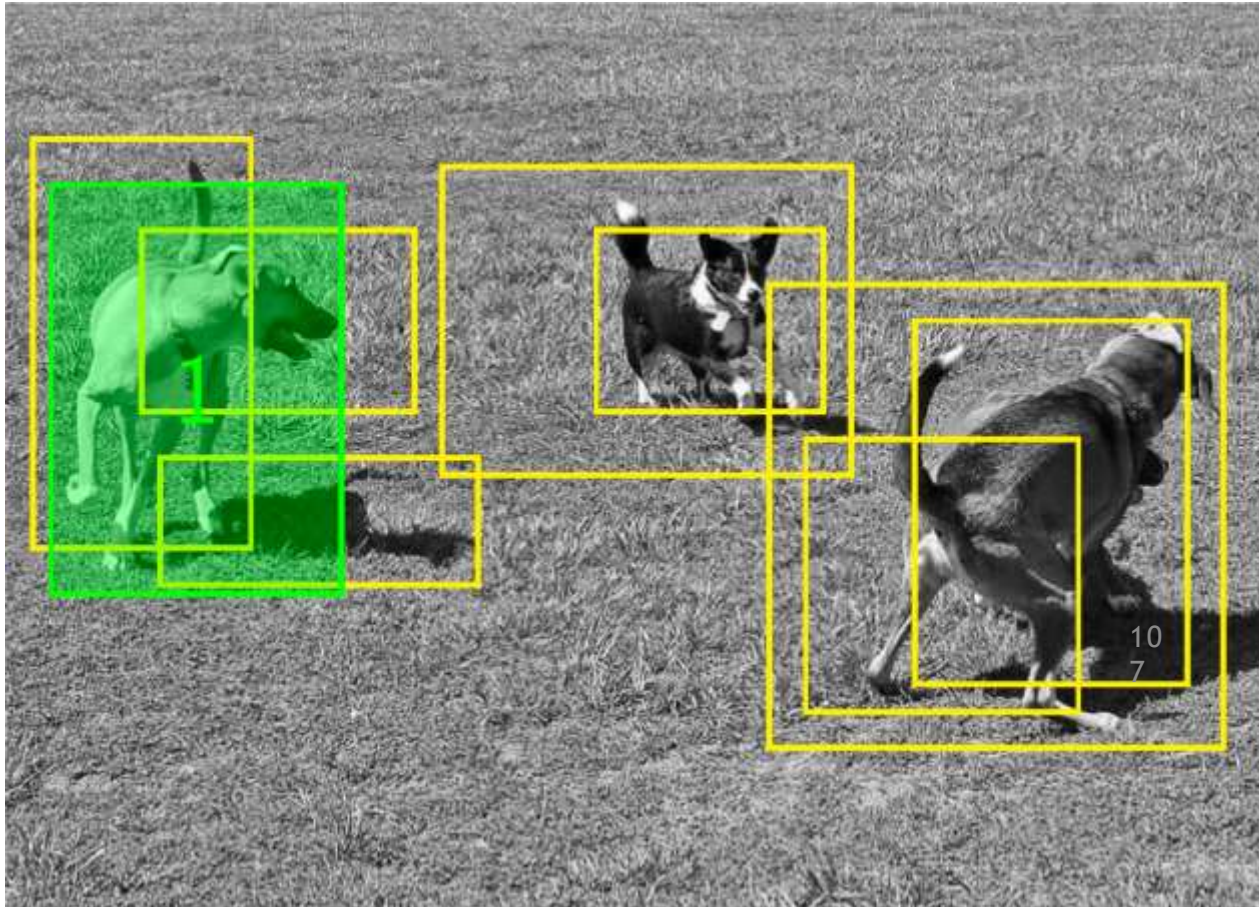
N M S

- Many bounding boxes are predicted by the CNN



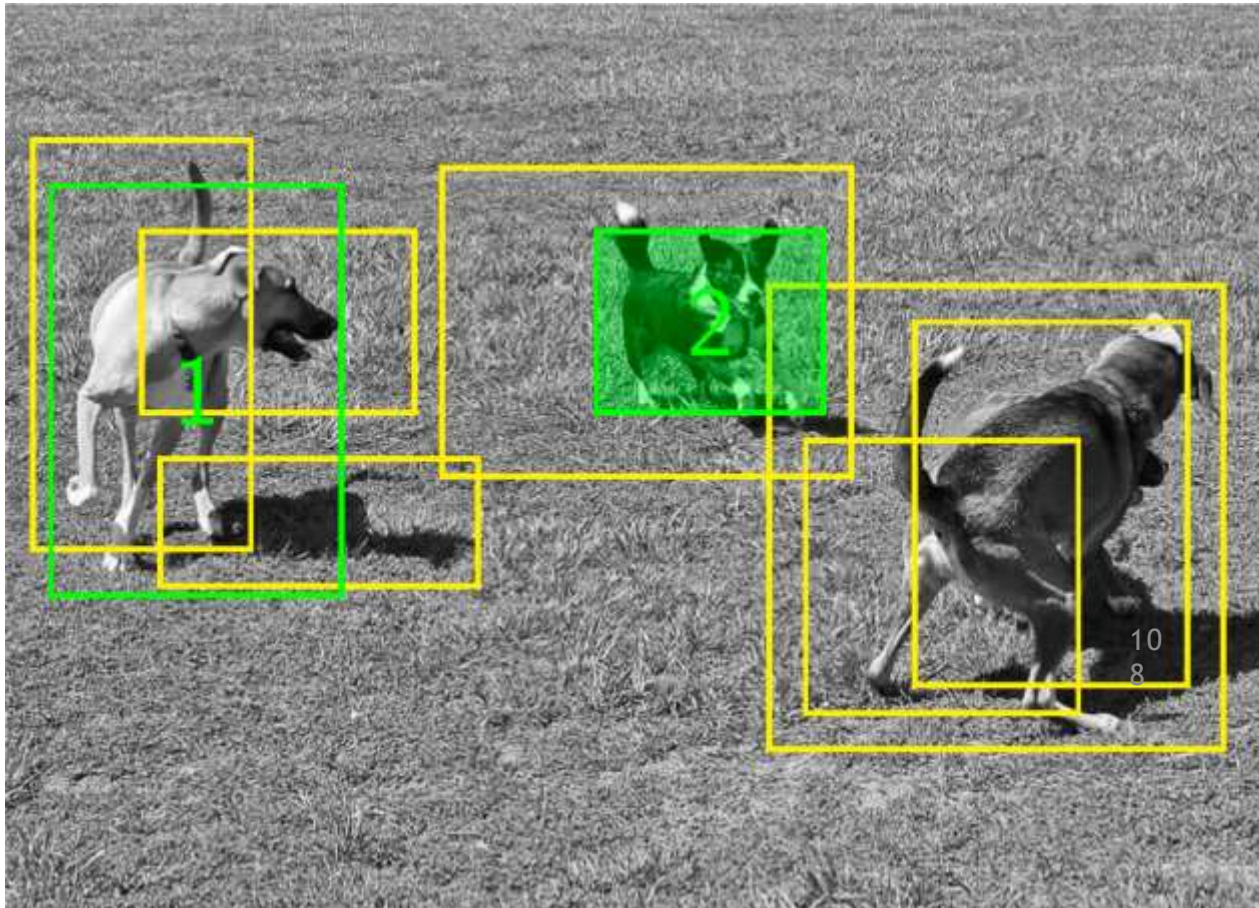
N M S

- Most confident region remains



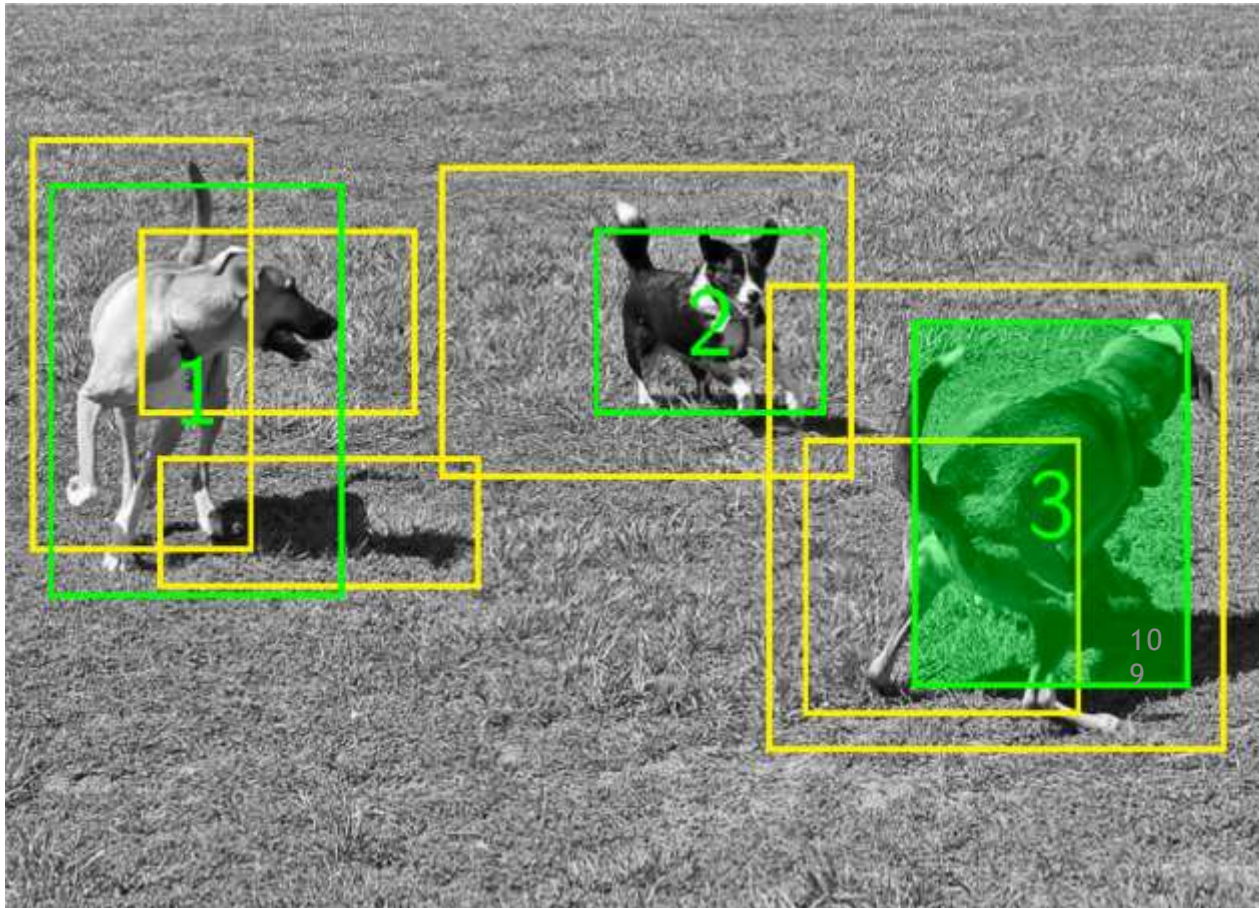
N M S

- Second most confident region remains



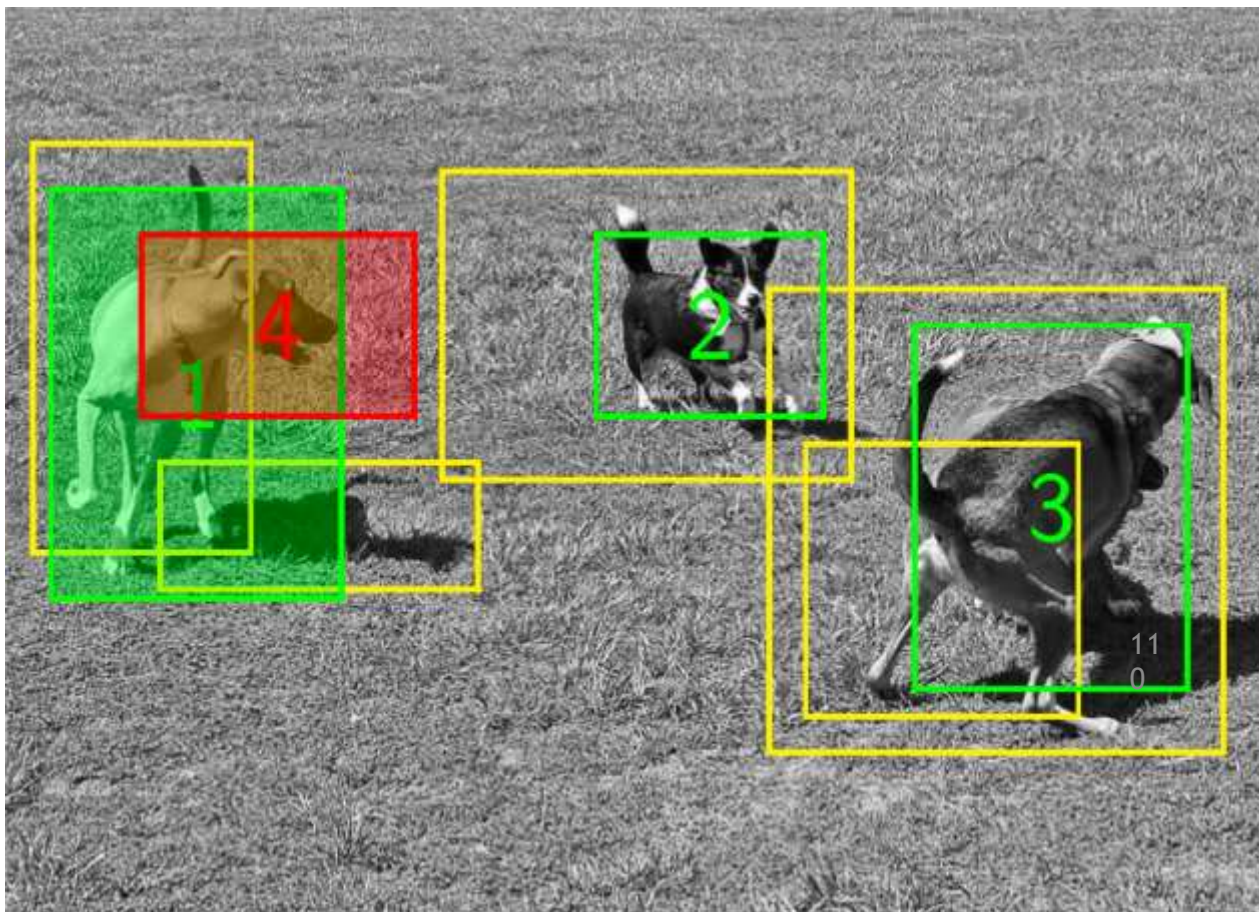
N M S

- Third most confident region remains



N M S

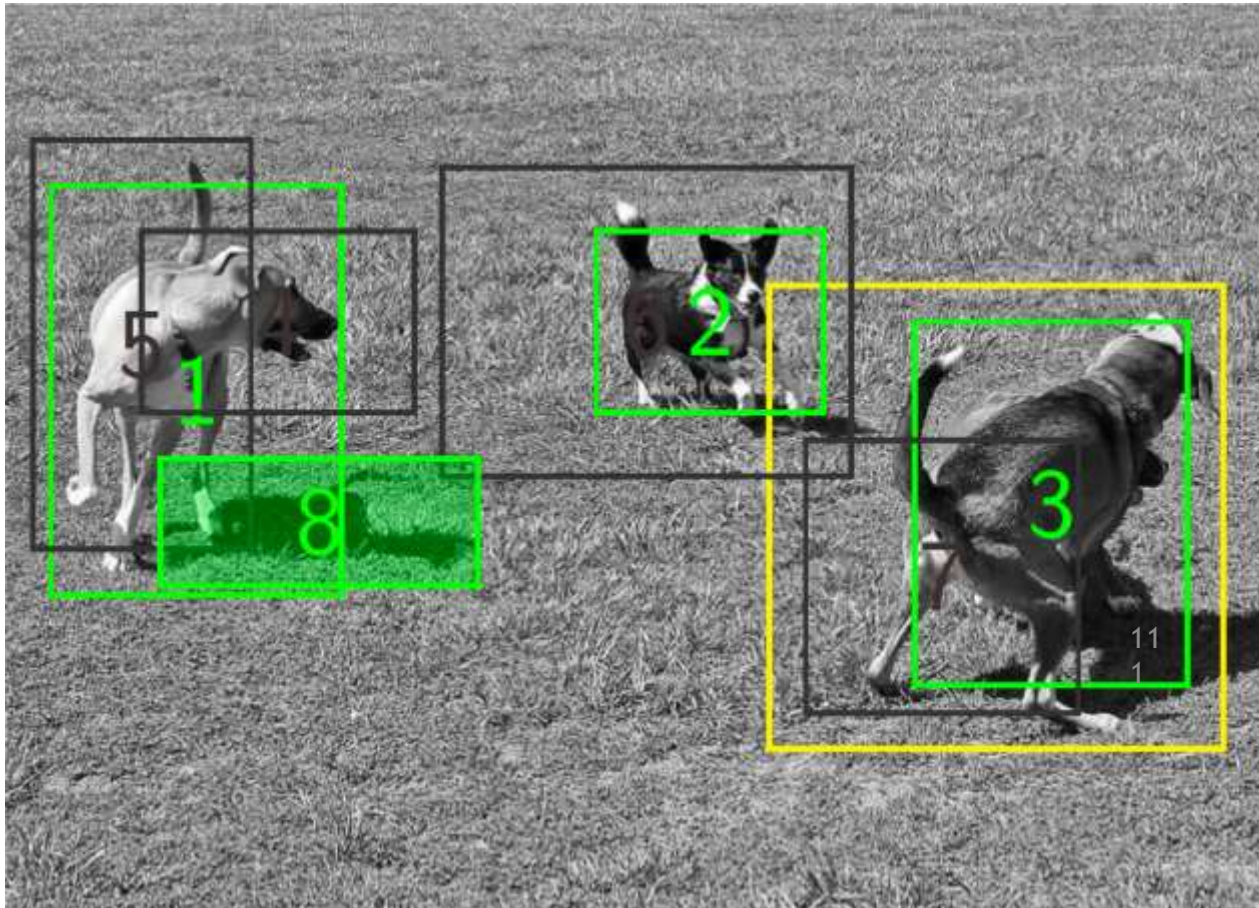
- Region 4 overlap with 1 more than threshold → remove



N M S

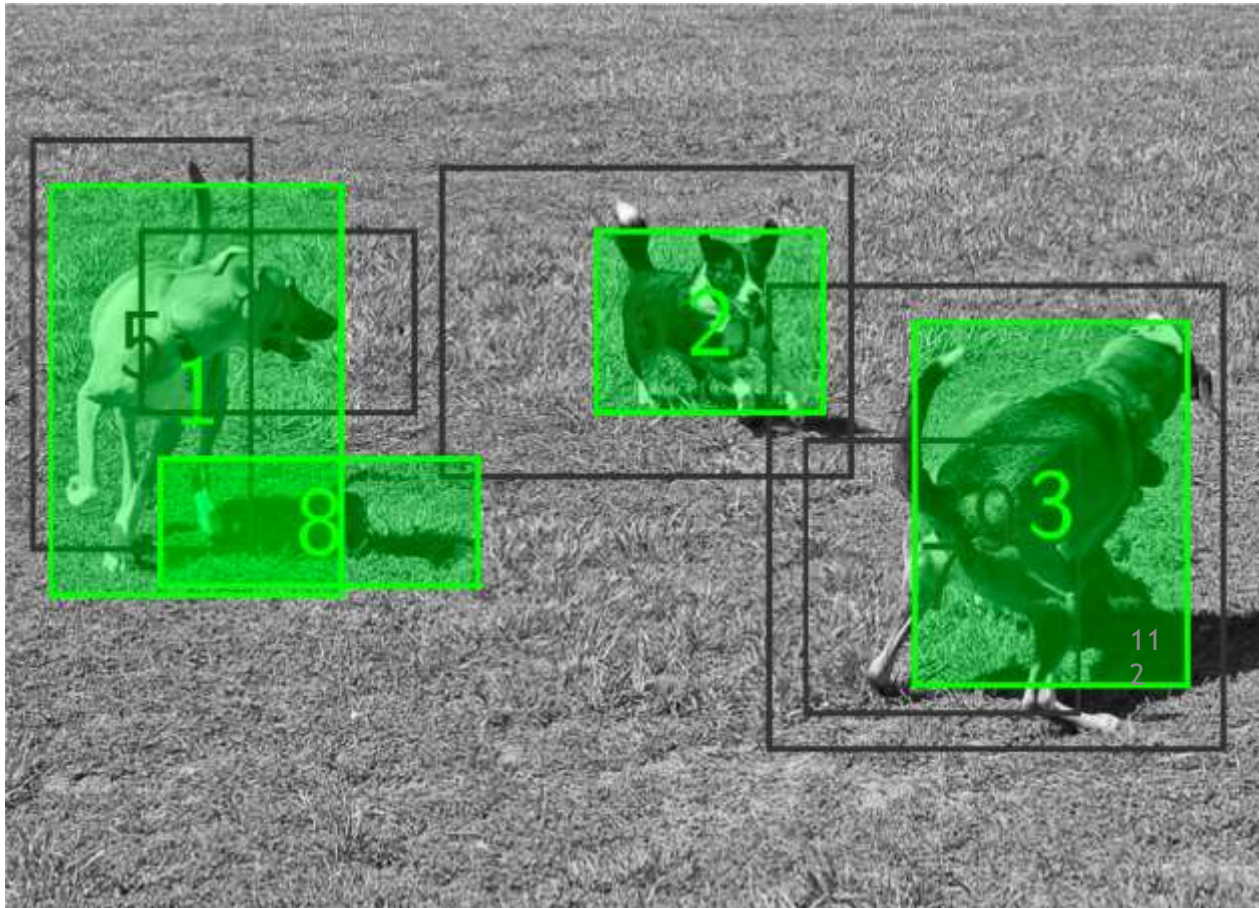
- The same will happen to all the regions in black

Region 8 does not overlap with green boxes (confirmed detections), hence we keep it.



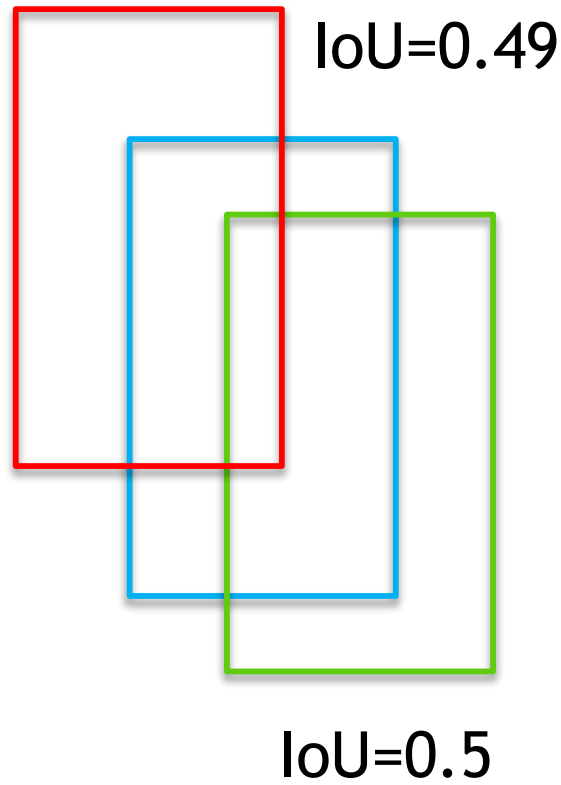
N M S

- In the end, we have 4 confirmed predicted boxes.



N M S

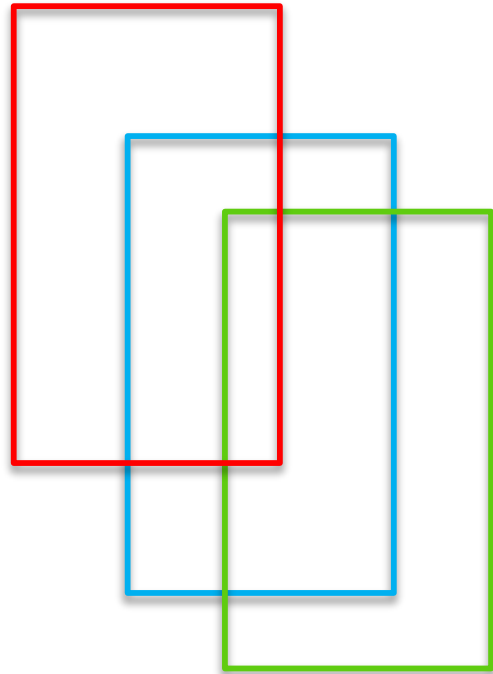
- Problem with highly overlapping objects



With an NMS threshold of 0.4, we remove both boxes, hence we have a False Negative.

N M S

IoU=0.49



IoU=0.5

- Problem with highly overlapping objects
- With an NMS threshold of 0.6, we keep both boxes, hence we have a False Positive

Object Detection Datasets



PASCAL

20 categories
6k training images
6k validation images
10k test images



COCO

80 categories
200k training images
60k val + test images



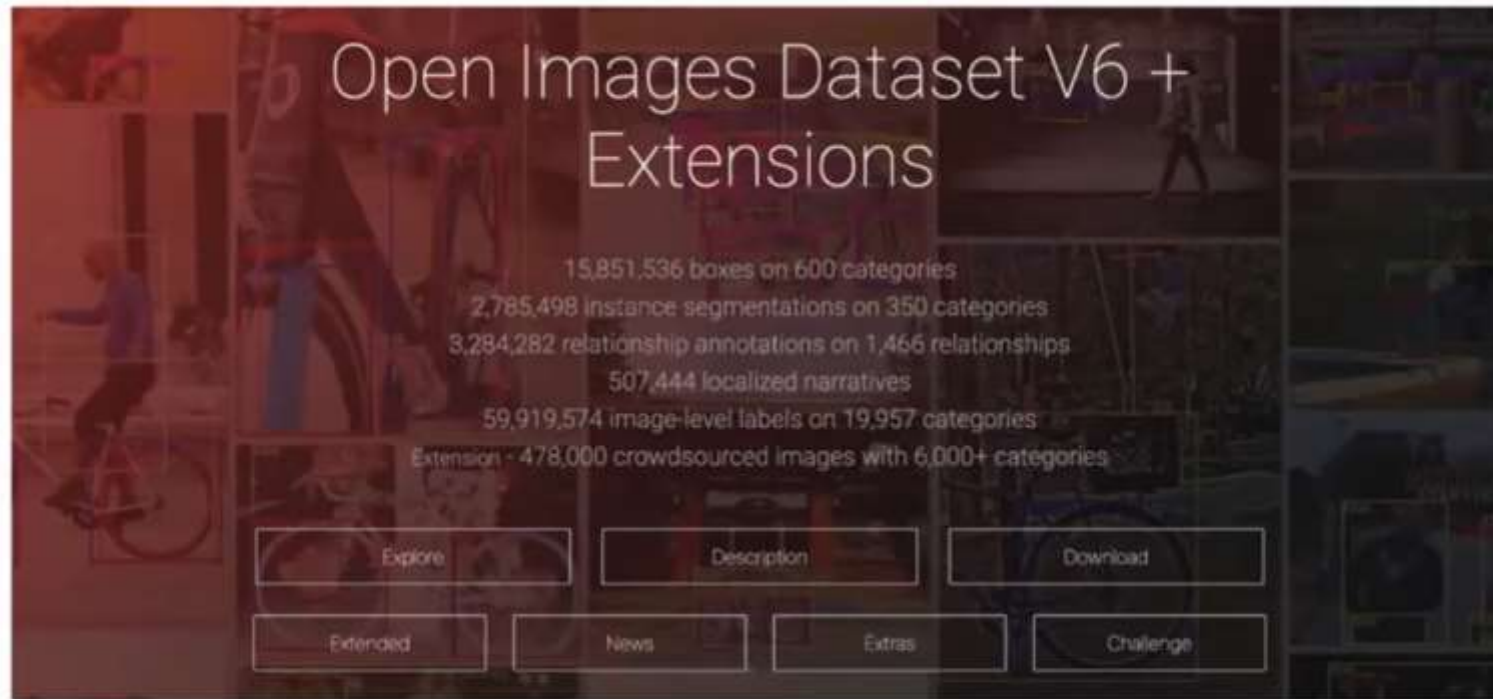
ILSVRC

200 categories
456k training images
60k validation + test images

Object Detection Datasets



Object Detection Datasets



Kuznetsova, A., Rom, H., Alldrin, N., Uijlings, J., Krasin, I., Pont-Tuset, J., ... & Ferrari, V. [The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale](#). IJCV 2020. [\[dataset\]](#)

General Object Detection datasets

- **PASCAL VOC**

- 20 object categories
- Classic benchmark dataset
- Simpler compared to COCO but still widely **referenced**

- **Open Images Dataset (Google)**

- Thousands of object classes
- Millions of bounding box annotations
- Rich hierarchical label structure

- **ImageNet Detection**

- Detection subset of ImageNet

Remote Sensing / Aerial Image Datasets

- Used for satellite and aerial object detection, often with rotated bounding boxes (OBB).

DOTA (Dataset for Object Detection in Aerial Images)

- Oriented bounding box (OBB) annotations
- Categories: planes, ships, vehicles, buildings, etc.
- Versions: v1.0, v1.5, v2.0

<https://docs.ultralytics.com/datasets/obb/dota-v2/#dota-v20>